
UNIT 2 PROBLEM SOLVING USING SEARCH

Structure	Page No
2.0 Introduction	43
2.1 Objectives	46
2.2 Introduction to State Space Search	46
2.2.1 Problem Formulation	47
2.2.2 Structure of a State space	47
2.2.3 Problem solution of State space	47
2.3.4 Searching for solution in state spaces formulated	48
2.3 Formulation of 8 puzzle problem from AI perspective	50
2.4 N-queen's problem- Formulation and Solution	52
2.4.1 Formulation of 8 Queen's problem	55
2.4.2 State space tree for 4-Queen's problem	55
2.4.3 Backtracking approach to solve N Queen's problem	57
2.5 Two agent search: Adversarial search	60
2.5.1 Elements of Game playing search	60
2.5.2 Types of algorithms in Adversarial search	62
2.6 Minimax search strategy	63
2.6.1 Minimax algorithm	63
2.6.2 Working of Minimax algorithm	64
2.6.3 Properties of Minimax algorithm	67
2.6.4 Advantages and Disadvantages of Minimax search	67
2.7 Alpha-Beta Pruning algorithm	68
2.7.1 Working of Alpha-Beta pruning	68
2.7.2 Move Ordering of Alpha-Beta pruning	77
2.8 Summary	80
2.9 Solutions/Answers	82
2.10 Further readings	90

2.0 INTRODUCTION

Many AI-based applications need to figure out how to solve problems. In the world, there are two types of problems. First, the problem which can be solved by using a deterministic procedure, and the success is guaranteed. But most real-world problems can be solved only by searching for a solution. AI is concerned with these second types of problems solving.

To build a system to solve a problem, we need to

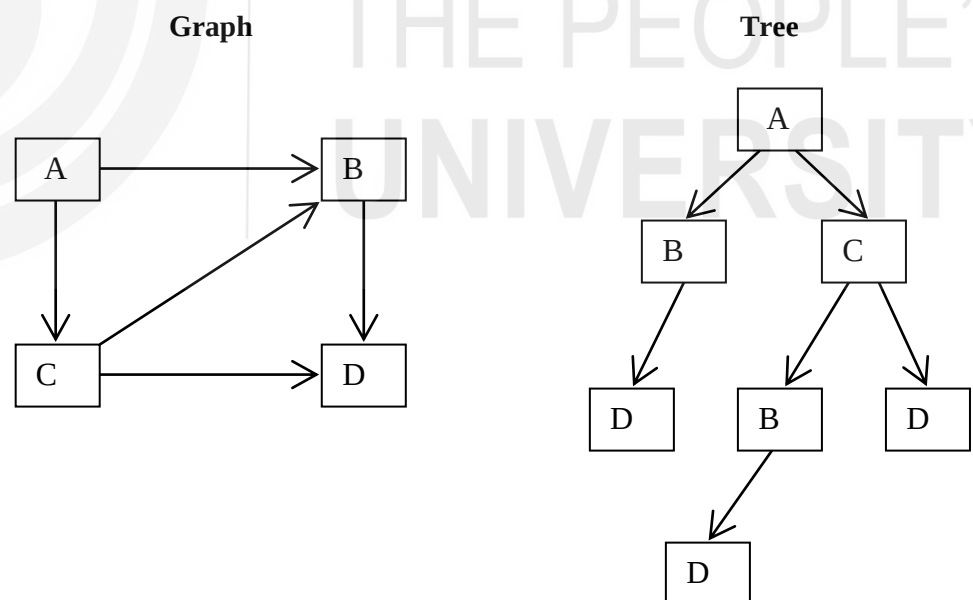
- **Define the problem precisely** - find initial and final configuration for an acceptable solution to the problem.
- **Analyse the problem** - find few important features that may have an impact on the appropriateness of various possible techniques for solving the problem
- **Isolate and represent** task knowledge necessary to solve the problem
- **Choose the best problem-solving technique(s)** and apply it to the particular problem.

To provide a formal description of a problem, we need to do the following:

- Define a **state space** that contains all the possible configurations of the relevant objects.
- Specify one or more states that describe possible situations, from which the problem-solving process may start. These states are called **initial states**.
- Specify one or more than one **goal states**.
- Defining a *set of rules* for the actions (operators) that can be taken.

The problem can then be solved by using the rules, in combination with an appropriate **control strategy**, to move through the problem space until a path from an **initial state** to a **goal state** is found. This process is known as '**search**'. Thus, search is fundamental to the problem-solving process. Search is a general mechanism that can be used when a more direct method is not known. Search provides the framework into which more direct methods for solving subparts of a problem can be embedded. **All AI problems are formulated as search problems.**

A **problem space** is represented by a directed graph, where *nodes* represent *search states*, and *paths* represent the operators that change the state. To simplify search algorithms, it is often convenient to logically and programmatically represent a problem space as a **tree**. A tree usually decreases the complexity of a search at a cost. Here, the cost is due to duplicating some nodes on the tree that were linked numerous times in the graph, e.g., node B and node D shown in the example below.



A **tree** is a graph in which any two vertices are connected by exactly one path. Alternatively, any connected graph with no cycles is a tree.

Before an AI problem can be solved, it must be represented as a **state space**. Here, **state** means representation of elements at a given moment. Among all possible states, there are two special states called the **initial state** (the start point) and the **final state** (the goal state). A **successor function (a set of operators)** is used to change the state. It is used to move from one state to another. A state space is the set of all states reachable from the initial state. A

state space essentially consists of a set of nodes representing each state of the problem, arcs between nodes representing the legal moves from one state to another, an initial state, and a goal state. Each state space takes the form of a tree or a graph. In AI, a wide range of problems can be formulated as a search problem. The process of searching means a sequence of actions that takes you from an initial state to a goal state, as shown in the following figure 1.

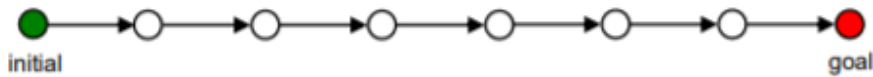


Fig 1: A sequence of actions in a search space from the initial to the goal state.

So, the state space is one of the methods to represent the problem in AI. A set of all possible states reachable from the initial state by taking some sequence of actions (using some operator) for a given problem is known as the **state space** of the problem. A state space represents a problem in terms of **states** and **operators** that change states”.

In this unit, we examine the concept of a **state space** and the different **search processes** that can be used to explore the search space in order to find a solution (Goal) state. In the worst case, search explores all possible paths between the *initial state* and the *goal state*.

For a better understanding of these definitions described above, consider the following 8-puzzle problem:

Eight-Puzzle problem formulation from AI perspective:

initial state: some configuration of the 8-tiles on a 9-cell board.

operators (Action): It’s easier if we focus on the blank. There are 4 operators that are: “Moving the blank”: UP, DOWN, LEFT, and RIGHT.

Uniformed/Blind 3	1	2		3		2
4		5		4	1	5
6	7	8	the blank up	6	7	8

Goal state: Tiles in a specific order

	1	2
3	4	5
6	7	8

Solution: Optimal sequence of operators (Actions)

Path costs:

Cost of each action = 1

Cost of a sequence of actions = the number of actions

A state space representation of the 8-puzzle problem is shown in figure-2

8-Puzzle

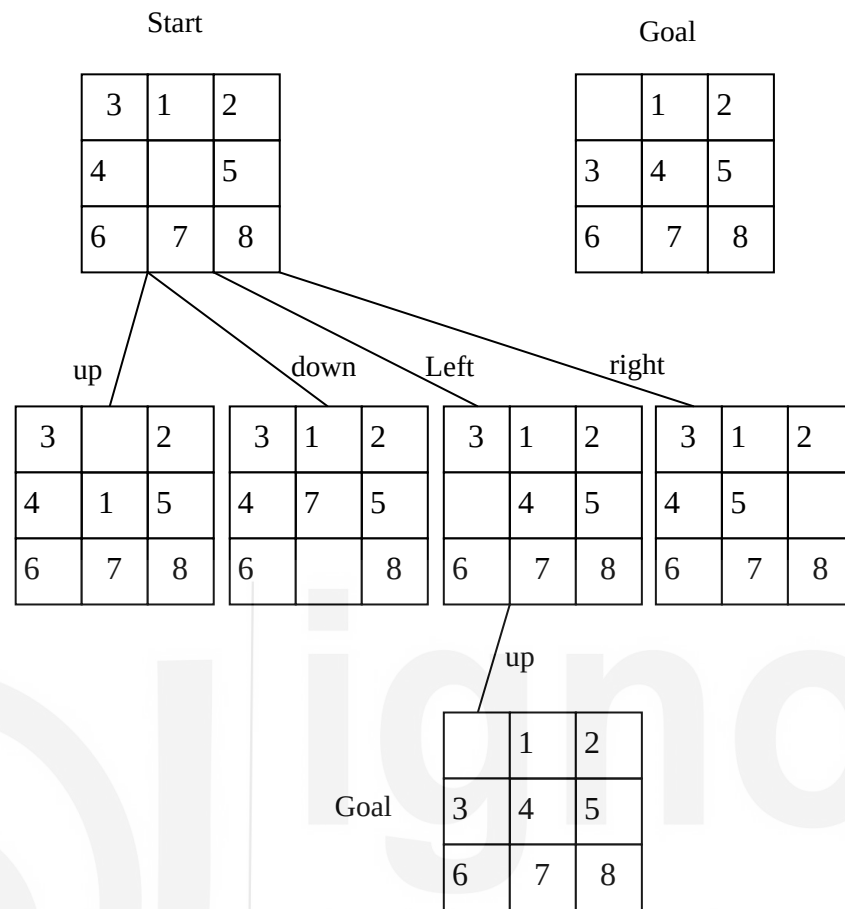


Fig. 2: A state space representation of the 8-puzzle problem generated by “Move Blank” operator

2.1 OBJECTIVES

After studying this unit, you should be able to:

- Understand the state space search
- Formulate the problems in the form of a state space
- Understand how implicit state space can be unfolded during search
- Explain the state space search representation for the Water-Jug, 8-puzzle, and N-Queens problem.
- Solve N Queen’s Problem using Backtracking approach
- Understand adversarial search (two-agent search)
- Differentiate between the Minimax and Alpha-beta pruning search algorithms.

2.2 INTRODUCTION TO STATE SPACE SEARCH

It is necessary to represent an AI problem in the form of a state space before it can be solved. A state space is the set of all states reachable from the initial state. A state space forms a graph in which the nodes are states, and the arcs

between nodes are actions. In the state space, a path is a sequence of states connected by a sequence of actions.

2.2.1 Structure of a state space:

The structures of state space are trees and graphs. A tree has one and only one path from any point to any other point. A graph consists of a set of nodes (vertices) and a set of edges (arcs). Arcs establish relationships (connections) between the nodes, i.e., a graph has several paths to a given node. **Operators** are directed arcs between nodes.

The method of solving problems through AI involves the process of defining the search space, deciding start and goal states, and then finding the path from the start state to the goal state through the search space.

The search process explores the state space. In the worst case, the search explores all possible paths between the initial state and the goal state.

2.2.2 Problem Solution:

In a state space, a **solution is a path** from the initial state to a goal state or sometimes just a goal state. A numeric cost is assigned to each path. It also gives the cost of applying the operators to the states. A path cost function is used to measure the quality of a solution, and out of all possible solutions, an optimal solution has the lowest path cost. The importance of cost depends on the problem and the type of solution required.

2.2.3 Problem formulation:

Many problems can be represented as a **state space**. The state space of a problem includes: an *initial state*, one or more *goal states*, a set of *state transition operators* (or a **set of production rules**), used to change the current state to another state. This is also known as **actions**. A **control strategy** is used that specifies the order in which the rules will be applied. For example, Depth-first search (DFS), Breadth-first search (BFS), etc. It helps to find the goal state or a path to the goal state.

In general, a state space is represented by 4 tuples as follows: $S_s: [S, s_0, O, G]$

Where **S**: Set of all possible states.

s_0 : start state (initial configuration) of the problem, $s_0 \in S$.

O: Set of production rules (or set of state transition operators) used to change the state from one state to another. It is the set of arcs (or links) between nodes.

The **production rule** is represented in the form of a pair. Each pair consists of a left side that determines the applicability of the rule and a right side that describes the action to be performed, if the rule is applied.

G: Set of Goal states, $G \in S$.

The sequence of actions (or operators) is called a solution path. It is a path from the initial state to a goal state. This sequence of actions leads to a number of states, starting from the initial state to a goal state, as $\{s_0, s_1, s_2, \dots, s_n \in G\}$. A sequence of states is called a path. The cost of a path is a positive number. In most of the cases, the path cost is computed as the sum of the costs of each action.

The following Figure 3 shows a search process in a given state space.

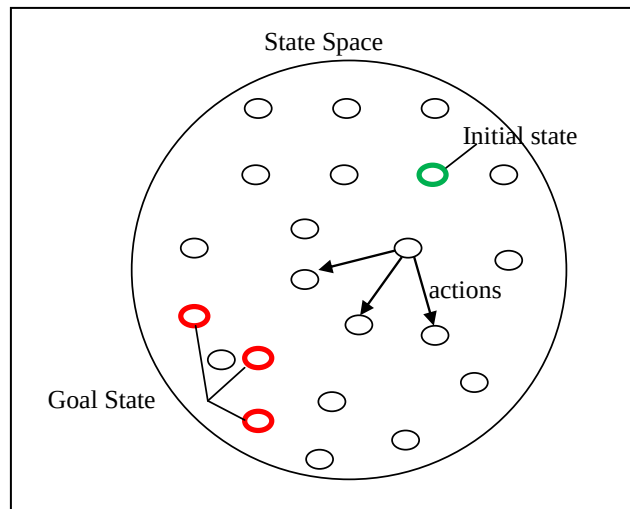


Fig. 3: State space with initial and goal node

We need to identify a sequence of actions that will turn the initial state s_0 into the desired goal state G . State space is commonly defined as a directed graph or as a tree in which each node is a state and each arc represents the application of an operator transforming a state to a successor state.

2.2.4 SEARCHING FOR SOLUTIONS

After formulating our problem, we are ready to solve it. This can be done by searching through the state space for a solution. This search will be applied on a search tree or, generally, a graph that is generated using the initial state and the successor function.

Searching is applied to a search tree generated through state expansion. State expansion applies the successor function to the current state, where each state represents a node in the tree.

Generally, search is about selecting an option and putting the others aside for later in case the first option does not lead to a solution. The choice of which option to expand first is determined by the search strategy used.

Thus, the problem is solved by using the rules (operators), in combination with an appropriate control strategy, to move through the problem space until a path from the **initial state to a goal state** is found. This process is known as **search**. A solution path is a path in the state space from s_0 (initial state) to G (Goal state).

Example 1: State space representation of the Water-Jug problem (WJP):

Problem statement:

Given two jugs, a 4-gallon and a 3-gallon, both of which do not have measuring indicators on them. The jugs can be filled with water with the help of a pump that is available (as shown in Figure 4)

The question is “how can you get exactly 2 gallons of water into a 4-gallon jug?”

The Water Jug Problem

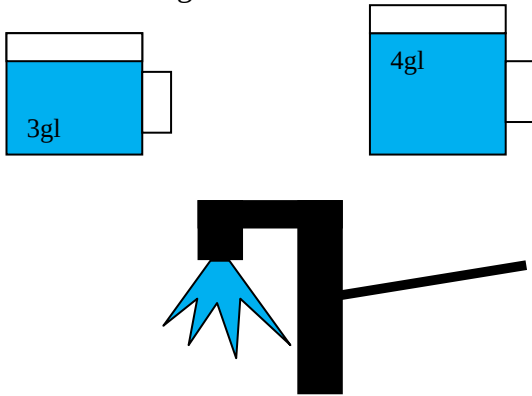


Fig. 4: The Water Jug problem

The state space or production rule of this problem can be defined as a collection of ordered pairs of integers (x, y) where $x = 0, 1, 2, 3, 4$ and $y = 0, 1, 2, \text{ or } 3$.

In the order pair (x, y) , x is the amount of water in a four-gallon jug, and y is the amount of water in the three-gallon jug.

State space: all possible combinations of (x, y)

The start state is $(0,0)$ and

Goalstate is $(2, n)$, where $n = 0, 1, 2, 3$

The following table-1 shows the set of **production rules** (actions) that can be used to change one state to another.

Table-1: Production rules for the Water Jug problem

Production Rules		
Rule No	Production	Meaning
R1	$(x, y \mid x < 4) \rightarrow (4, y)$	Fill 4-gallon jug
R2	$(x, y \mid y < 3) \rightarrow (x, 3)$	Fill 3-gallon jug
R3	$(x, y \mid x > 0) \rightarrow (0, y)$	Empty 4-gallon jug
R4	$(x, y \mid y > 0) \rightarrow (x, 0)$	Empty 3-gallon jug
R5	$(x, y \mid x + y \geq 4 \text{ and } y > 0) \rightarrow (4, y - (4 - x))$	Pour water from 3-gallon jug into 4-gallon jug until 4-gallon jug is full
R6	$(x, y \mid x + y \geq 3 \text{ and } x > 0) \rightarrow (x - (3 - y), 3)$	Pour water from 4-gallon jug into 3-gallon jug until 3-gallon jug is full
R7	$(x, y \mid x + y \leq 4 \text{ and } y > 0) \rightarrow (x + y, 0)$	Pour all water from 3-gallon jug into 4-gallon jug.
R8	$(x, y \mid x + y \leq 3 \text{ and } x > 0) \rightarrow (0, x + y)$	Pour all water from 4-gallon jug into 3-gallon jug.
R9	$(x, y \mid x > 0) \rightarrow (x - d, y)$	Pure some water d out from 4-gallon jug
R10	$(x, y \mid y > 0) \rightarrow (x, y - d)$	Pure some water d out from 3-gallon jug

The following 2 solutions are found for the problem “how can you get exactly 2 gallons of water into 4-gallon jug”, as shown in Table-2 and in Table-3.

Solution-1:

Table 2: Getting exactly 2 gallons of water into 4-gallon jug (solution1)

4-gal jug	3-gal jug	Rule applied
0	0	Initial state
4	0	R1 {fill 4-gal jug}
1	3	R6 {Pure all water from 4-gal jug to 3-gal jug}
1	0	R4 {empty 3-gal jug}
0	1	R8 {Pour water from 4 to 3-gal jug}
4	1	R1 {Fill 4-gal jug}
2	3	R6 {Pour water from 4-gal jug to 3-gal jug until it is full}
2	0	R4 {empty 3-gal jug}

Solution-2

Table 3: Getting exactly 2 gallons of water into 4-gallon jug (solution2)

4-gal jug	3-gal jug	Rule applied
0	0	Initial state
0	3	R2 {fill 3-gal jug}
3	0	R7 {Pure all water from 3-gal jug to 4-gal jug}
3	3	R2 {fill 3-gal jug}
4	2	R5 {Pour from 3 to 4-gal jug until it is full}
0	2	R3 {Empty 4-gal jug}
2	0	R7 {Pour all water from 3-gal jug to 4-gal jug}

A state space tree for WJP with all possible solutions is shown in Figure 5.

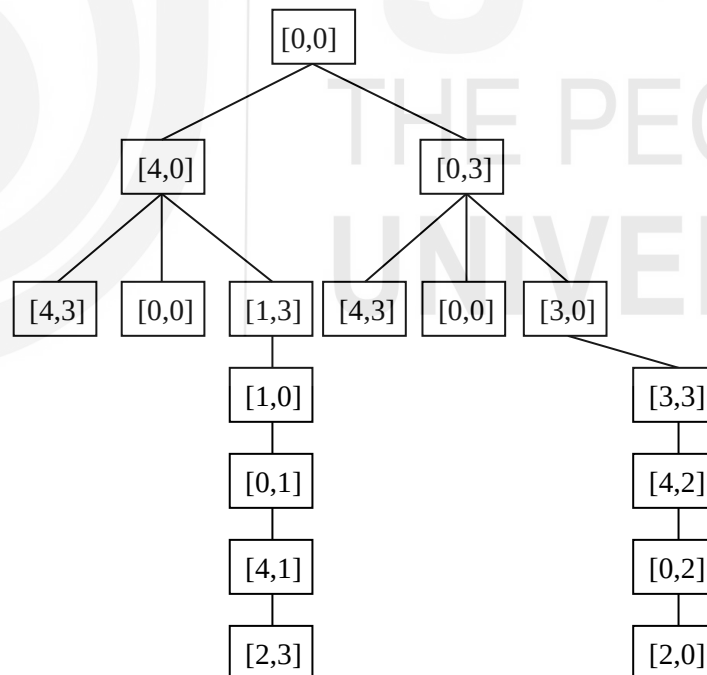


Fig 5: All possible solutions of WJP using the state space tree

2.3 FORMULATION OF THE 8 PUZZLE PROBLEM

The eight-tile puzzle consists of a 3-by-3 (3×3) square frame board that holds eight (8) movable tiles numbered as 1 to 8. One square is empty, allowing the

adjacent tiles to be shifted. The objective of the puzzle is to find a sequence of tile movements that leads from a starting configuration to a goal configuration

The Eight Puzzle Problem formulation:

Given a 3×3 grid with 8 sliding tiles and one “blank.”

Initial state: some other configuration of the tiles, for example

3	1	2
4		5
6	7	8

Goal state:

	1	2
3	4	5
6	7	8

Operator: Slide tiles (Move Blank) to reach the goal (as shown below). There are 4 operators that are “Moving the blank”:

Move the blank UP,

Move the blank DOWN,

Move the blank LEFT and

Move the blank RIGHT.

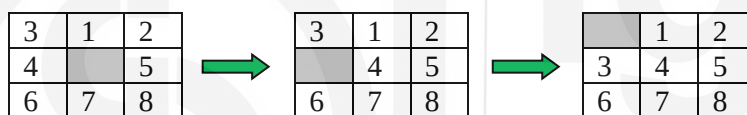


Fig. 6: Moving Blank LEFT and then UP

Path Cost: Sum of the cost of each path from the initial state to the goal state. Here, the cost of each action (blank move) = 1, so the cost of a sequence of actions = the number of actions. An optimal solution has the lowest cost path.

Performing State-Space Search: Basic idea:

If the initial state is a goal state, return it.

If not, apply the operators to generate all states that are one step from the initial state (its successors)

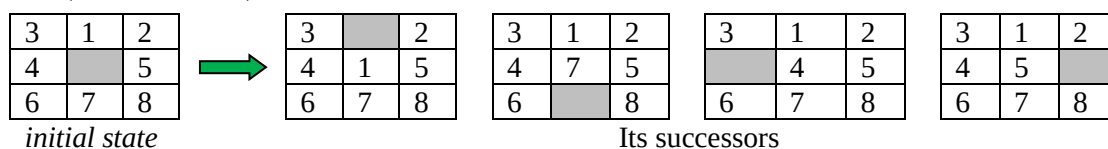


Fig. 7: All possible successors for a given initial state

Consider the successor (and their successors...) until you find a goal state.

Different search strategies consider the state in different orders. They may use different data structures to store the states that have yet to be considered.

State-Space Search Tree for 8-Puzzle problem:

The predecessor reference connects the search nodes, creating a data structure known as a tree.

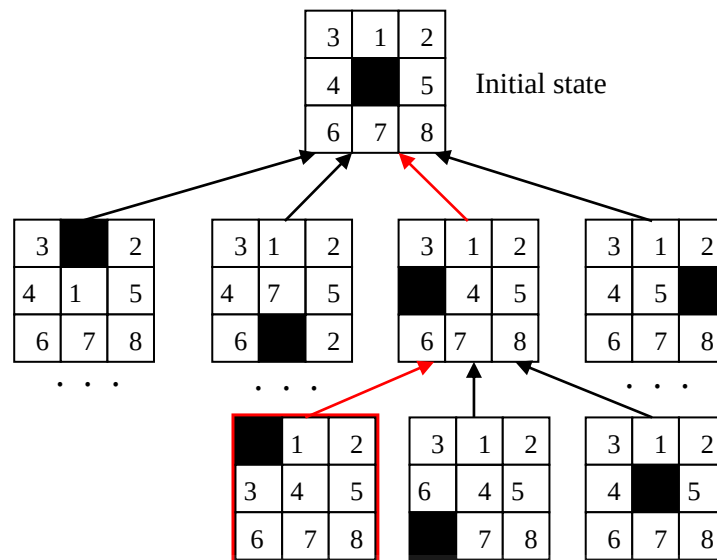


Fig. 8: Tracing a tree bottom-up from the goal state to the initial state

When we reach a goal, we trace up the tree to get the solution, i.e., the sequence of actions from the initial state to the goal.

Q.1 Find the minimum cost path for the 8-puzzle problem, where the start and goal state are given as follows:

1	2	3
4	8	-
7	6	5

Start State

1	2	3
4	5	6
7	8	-

Goal State

Initial State: $\{(1, 2, 3), (4, 8, -), (7, 6, 5)\}$

Successor State: $\{(1, 2, 3), (4, 8, 5), (7, 6, -)\}$; Move 5 up or – to down

$\{(1, 2, 3), (4, 8, 5), (7, -, 6)\}$; Move 6 right or – to left

$\{(1, 2, 3), (4, -, 5), (7, 8, 6)\}$; Move 8 down or – to up

$\{(1, 2, 3), (4, 5, -), (7, 8, 6)\}$; Move 5 to left or – to right

Goal State: $\{(1, 2, 3), (4, 5, 6), (7, 8, -)\}$; Move 6 up or – to down

PATH COST=5

2.4 N QUEEN'S PROBLEM-FORMULATION AND SOLUTION

The N-Queen problem is the problem of placing N Queens (Q1, Q2, Q3, ..., Qn) on an $N \times N$ chessboard so that no two queens attack each other. The colour of the queens is meaningless in this puzzle, and any queen is assumed to attack any other. So, a solution requires that no two queens share the same row, column, or diagonal.

The N-queen problem must follow the following rules:

1. There is at most one queen in each column.
2. There is at most one queen in each row.
3. There is at most one queen in each diagonal.

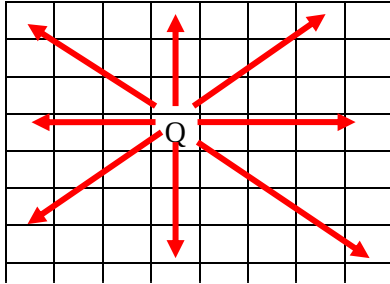


Fig. 9: No two queens placed on the same row, column, or diagonal

The N Queen's problem was originally proposed in 1848 by the chess player Max Bazzel, and over the years, many mathematicians, including Gauss, have worked on this puzzle. In 1874, S. Gunther proposed a method of finding solutions by using determinants, and J.W.L. Glaisher refined this approach.

The solutions that differ only by summary operations (rotations and reflections) of the board are counted as one. For 4 queens' problems, there are 16_{C_4} possible arrangements on a 4×4 chessboard and there are only 2 possible solutions for 4 Queen's problem. Note that there is only 1 unique solution, out of 2 possible solutions, as the second solution is just a mirror image of the first solution

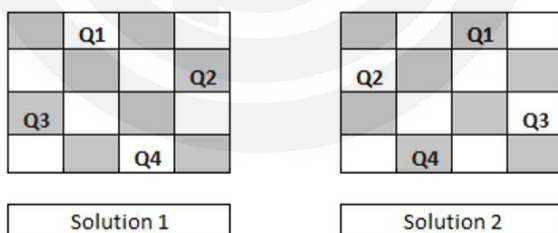


Fig 10 Two possible solutions of 4-Queen's problem

Similarly, the one possible solution for the 8-queens problem is shown in Figure 11.

The 8-queen problem is computationally very expensive since the total number of possible arrangements of queens on a 8×8 chessboard is $64_{C_8} = 64!/(56! \times 8!) \approx 4.4 \times 10^9$. Note that the 8-Queens problem has 92 **distinct** solutions and 12 **unique** solutions, as shown in table-4.

		column →							
		1	2	3	4	5	6	7	8
Row	1				Q				
	2						Q		
	3								Q
	4		Q						
	5							Q	
	6	Q							
	7			Q					
	8					Q			

Fig. 11: One possible solution of 8 Queens problem

8-tuple = (4, 6, 8, 2, 7, 1, 3, 5)

The following table-4 summarizes both distinct and unique solutions for the problem of 1-Queen to 26 Queens problem. In general, there is no known formula to find the exact number of solutions for the N-queens problem.

Table-4 Solution of N Queen's problem for N=1 to N=26, both Unique and Distinct

No. of Queen's	Unique Solution	Total distinct solutions
1	1	1
2	0	0
3	0	0
4	1	2
5	2	10
6	1	4
7	6	40
8	12	92
9	46	352
10	92	724
11	341	2,680
12	1787	14,200
13	9233	73,712
14	45,752	365,596
15	285,053	2,279,184
16	1,846,955	14,772,184
17	11,977,939	95,815,104
18	83,263,591	666,090,624
19	621,012,754	4,968,057,848
20	4,878,666,808	39,029,188,884
21	39,333,324,973	314,666,222,712
22	336,376,244,042	2,691,008,701,644
23	3,029,242,658,210	24,233,937,684,440
24	28,439,272,956,934	227,514,171,973,736
25	275,986,683,743,434	2,207,893,435,808,352
26	2,789,712,466,510,289	22,317,699,616,364,044

2.4.1 Formulation of 4-Queen's problem:

States: any arrangement of 0 to 4 queens on the board

Initial state: 0 queens on the board

Successor function: Add a queen in any square

Goal test: 4 queens on the board, none attacked

For the initial state, there are 16 successors. At the next level, each of the states has 15 successors, and so on down the line. This search tree can be restricted by considering only those successors where no queens are attacking each other. To do that, we have to check the new queen with all the other queens on the board. In this way, the answer is found at a depth of 4. For the sake of simplicity, you can consider the problem of 4-Queens and see how the 4-Queens problem is solved using the concept of "Backtracking".

2.4.2 State space tree for the 4 Queens' Problem

We place the queen row-by-row (i.e., Q_1 in row 1, Q_2 in row 2 and so on). Backtracking gives "all possible solutions". If you want an optimal solution, then go for Dynamic programming.

Let's see the Backtracking method. There are $16C_4$ ways to place a queen on a 4x4 chess board as shown in the following state space tree (Figure 12). In a tree, the value (i, j) means in the i^{th} row, j^{th} queen is placed.

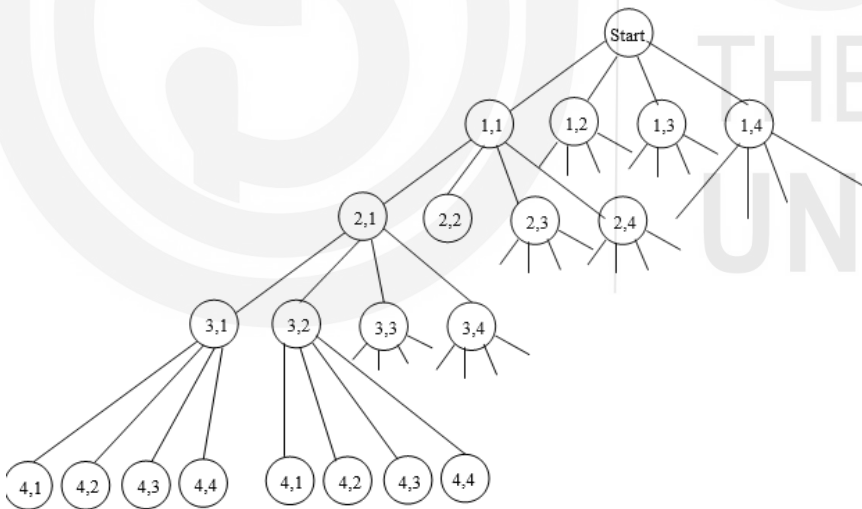


Fig. 12: State-space tree showing all possible ways to place a queen on a 4x4 chess board

So, to reduce the size (not anywhere on the chessboard, since there are $16C_4$ possibilities), we place queens row-by-row, and no Queen in the same column. This tree is called a permutation tree (**here we avoid same row or same column but allow diagonals**)

$$\text{Total nodes} = 1 + 4 + 4 \times 3 + 4 \times 3 \times 2 + 4 \times 3 \times 2 \times 1 = 65$$

The edges are labeled by possible values of x_i . Edges from level 1 to level 2 nodes specify the values for x_1 . Edges from the level i to level $i + 1$ are labeled with the values of x_i .

The solution space is defined by all paths from root node to leaf node. There are $4! = 24$ leaf nodes in the tree. Nodes are numbered as depth first Search. The state space tree for 4-Queen's problem (**avoid same row or same columns but allowing diagonals**) is shown in figure 13.

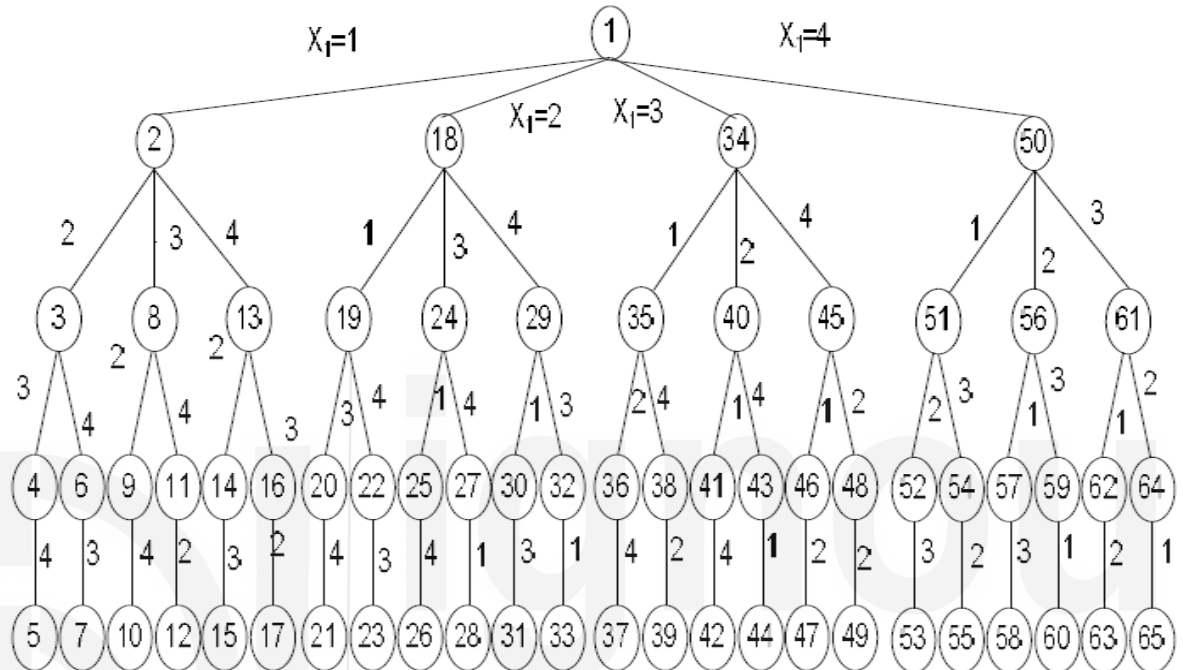


Fig. 13: State space tree for 4 queen's problems (allowing same diagonal but not same row and columnn)

The two solutions are found in the tree, as
 $(x_1, x_2, x_3, x_4) = (2, 4, 1, 3)$ and
 $(x_1, x_2, x_3, x_4) = (3, 1, 4, 2)$, which is shown in Figure 14.

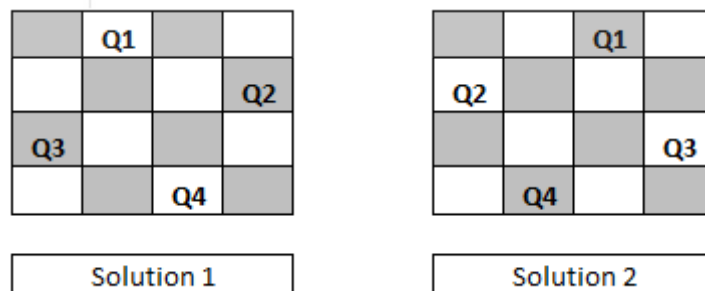


Fig. 14: Two possible solutions of 4-Queen's problem

Note that the second solution is just a mirror image of the first solution.

We can further reduce the search space, as shown in Figure 6, by avoiding the diagonal also. Now you can avoid the **same row**, **avoid same columns**, and **avoid same diagonals**, while placing any queen. In this case, the state space tree looks as shown in Figure 15.

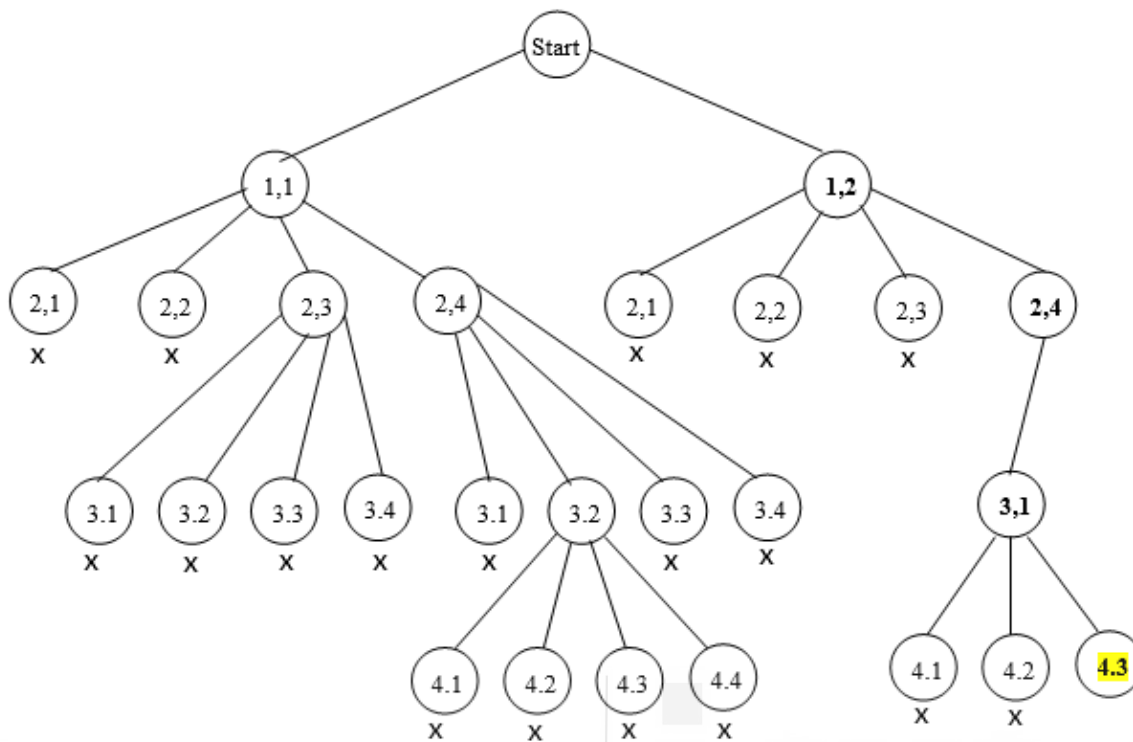


Fig 15: State space tree for 4 queen's problem (avoiding same row, columns, and diagonal)

Note that Queens are placed row-by-row, that is, Q_1 in row 1, Q_2 in row 2 and so on. In a tree, node (1,1) is a promising node (no queen attack) as Q_1 is placed in 1st row and 1st column. Node (2,1) is a non-promising node because we cannot place Q_2 in the same column (as Q_1 is already placed in column 1). Note that a non-promising node is marked as X. So, we try (2, 2) again, non-promising (due to same column), next try (2,3), it's a promising node, so proceed and try to place 3rd queen on 3rd row. But in 3rd row, all positions (3,1), (3,2), (3,3) and (3,4) are non-promising and we cannot place the Q_3 in any of this position. So, we backtrack to (1,1) and try for (2,4) and so on. Backtracking approach gives "all possible solutions". Figure 7 shows one possible solution for 4-Queen's problem as $\{(1,2), (2,4), (3,1), (4,3)\}$. This can also be written as $(x_1, x_2, x_3, x_4) = (2, 4, 1, 3)$. There are 2 possible solution of 4-Queen's problem. Another solution is $(x_1, x_2, x_3, x_4) = (3, 1, 4, 2)$, which is a mirror image of 1st solution.

2.4.3 Backtracking Approach to solve N queen's Problem:

Consider the chessboard squares' indices of the 2-Dimensional array $[1 \dots n, 1 \dots n]$. We observe that every element on the same diagonal that rows from the upper left to the right has the same $(row - column)$ value. It is called the **left diagonals**. Similarly, every element on the same diagonal that goes from the upper right to the lower left has same $(row + column)$ value. This is called **Right Diagonals**. For example, consider a 5×5 chessboard as $[1 \dots 5, 1 \dots 5]$ (as shown in figure 16).

Case 1: (Left diagonal):- Suppose queens are palced in same diagonal in locations: (1, 2), (2, 3), (3, 4), (4, 5) or (1, 4), (2, 5) or any other same left diagonal value. Observe that every element on the same diagonal has the same $(row - column)$ value. Similarly,

Case2: (right diagonal):- suppose queens are placed in same diagonal in locations: (1,3), (2,2), (3,1) or (1,4), (2,3), (3,2), (4,1) or any other same right diagonal value. Observe that every element on the same diagonal has the same $(row + column)$ value.

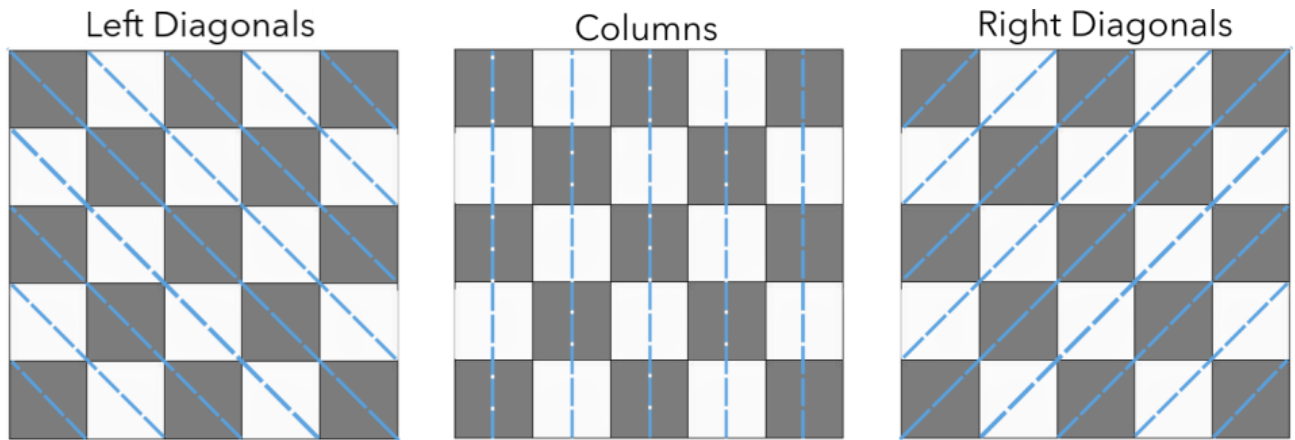


Fig. 16: Left diagonal and right diagonal for 5×5 chessboard.

Suppose two queens are placed at positions (i, j) and (k, l) then they are on the same diagonal if and only if:

$$(i - j) = (k - l) \text{ or } (j - l) = (i - k) \text{ -----(1) [left diagonal]}$$

$$(i + j) = (k + l) \text{ or } (j - l) = (k - i) \text{ ----- (2) [right diagonal]}$$

From equations (1) and (2), we can combine and write a one condition to check the diagonal as: **$abs(j - l) = abs(i - k)$** .

Algorithm NQueen (k, n)

// This procedure prints all possible placements of n queens on $n \times n$ chessboard so that they are non-attacking.

```
{
1. for i=1 to n do
2. {
3.   if place(k, i) then
4.   {
5.     x[k]=i;
6.     if (k==n) then print(x[1...n])
7.     else
8.       NQueen(k+1, n);
9.   }
10. }
11. }
```

Algorithm place(k, i)

/*This algorithm returns true if a queen can be placed in the kth row, ith column. Otherwise, it returns false. X[] is a global array. Abs® returns the absolute value of r */.

```

1. {
2.   for j=1 to k-1 do
3.     {
4.   if(x[j]=i)    // in the same column
5. Or (abs(x[j]-i)==abs(j-k))// in the
                        //same diagonal
6.       Return false;
7.     }
8.   Return true;
9. }

```

Check Your Progress 1

1. What are the various factors need to be taken into consideration when developing a state space representation?

.....

.....

2. Consider the following Missionaries and cannibals problem:
Three missionaries and three cannibals are on the side of a river, along with a boat that can hold one or two people. Find a way to get everyone to the other side, without ever leaving a group of missionaries outnumbered by cannibals.

- a) Formulate the missionaries and cannibals problem.
- b) Solve the problem formulated in part (a)
- c) Draw the state- space search graph for solving this problem.

.....

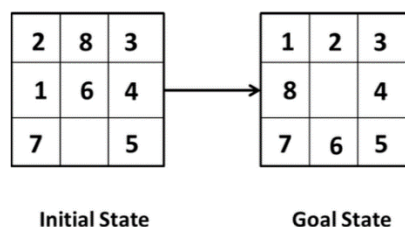
.....

3. Draw a state space tree representation to solve the Tower of Hanoi problem. (Hint: You can take the number of disks $n=2$ or 3).

.....

.....

4. Draw the state space tree for the following 8-puzzle problem, where the start and goal states are given below. Also, find the minimum cost path for this 8-puzzle problem. Each blank move is having cost=1.



.....

.....

5. Discuss a Backtracking algorithm to solve a N-Queen's problem. Draw a state space tree to solve a 4-Queen's problem.

.....

.....

2.5 ADVERSARIAL SEARCH-TWO AGENT SEARCH

In computer science, a search algorithm is an algorithm for finding an item with specified properties among a collection of items. The items may be stored individually as records in a database, or may be elements of a search space defined by a mathematical formula or procedure.

Adversarial search is a **game-playing** technique where the agents are surrounded by a competitive environment. A conflicting goal is given to the agents (multi-agent). These agents compete with one another and try to defeat one another in order to win the game. Such conflicting goals give rise to the adversarial search. Here, game-playing means discussing those games where **human intelligence** and **logic factor** is used, excluding other factors such as **luck**. **Tic-tac-toe, chess, checkers**, etc., are such types of games where no luck factor works; only the mind works.

Mathematically, this search is based on the concept of '**Game Theory.**' *According to game theory, a game is played between two players. To complete the game, one has to win the game, and the other loses automatically.'*



Techniques required to get the best optimal solution

There is always a need to choose those algorithms that provide the best optimal solution in a limited time. So, we use the following techniques, which could fulfil our requirements:

- **Pruning:** A technique that allows ignoring the unwanted portions of a search tree, which make no difference in its final result.
- **Heuristic Evaluation Function:** It allows to approximate the cost value at each level of the search tree, before reaching the goal node.

2.5.1 Elements of Game Playing search

To play a game, we use a game tree to know all the possible choices and to pick the best one. There are the following elements of a game-playing:

- **S₀:** It is the **initial state** from where a game begins.
- **PLAYER (s):** It defines which player is having the current turn to make a move in the state.
- **ACTIONS (s):** It defines the set of legal moves that a player can make to change the state.

- **RESULT (s, a):** It is a transition model that defines the result of a move.
- **TERMINAL-TEST (s):** Determines whether the game has ended. It returns true if state s is a terminal state, and false otherwise. States where the game is over are called terminal states.
- **UTILITY (s, p):** It defines the final value with which the game has ended. This function is also known as **the objective function** or **Payoff function**. This utility function gives a numeric value for the outcome of a game i.e.

For example, in **chess** and **tic-tac-toe**, we have two or three possible outcomes. Either win or lose, or draw the match, which we can represent by the values **+1, -1 or 0**. In other words, we can say that **(-1)**, if the PLAYER loses, **(+1)**, if the PLAYER wins, and **(0)**, If there is a draw between the PLAYERS.

Let's understand the working of the elements with the help of a game tree designed for **tic-tac-toe**. Here, the node represents the game state, and edges represent the moves taken by the players. The root of the tree is the initial state. Next level is all of MAX's moves, then next level is all of MIN's moves, and so on. Note that the root has 9 blank squares (MAX), level 1 has 8 blank squares (MIN), level 2 has 7 blank squares (MAX), and so on.

Objective:

Player1: Maximize outcome and **Player2:** Minimize outcome

Terminal (goal) state:

utility: -1, 0, +1 (that is, win for X is +1, win for O is -1, and 0 for a draw).

The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX. High values are assumed to be good for MAX and bad for MIN (which is how the players get their names). It is MAX's job to use the search tree to determine the best move. Note that if MAX wins, then the utility value is +1, if MIN wins, then the utility value is -1, and if DRAW, then the utility value is 0.

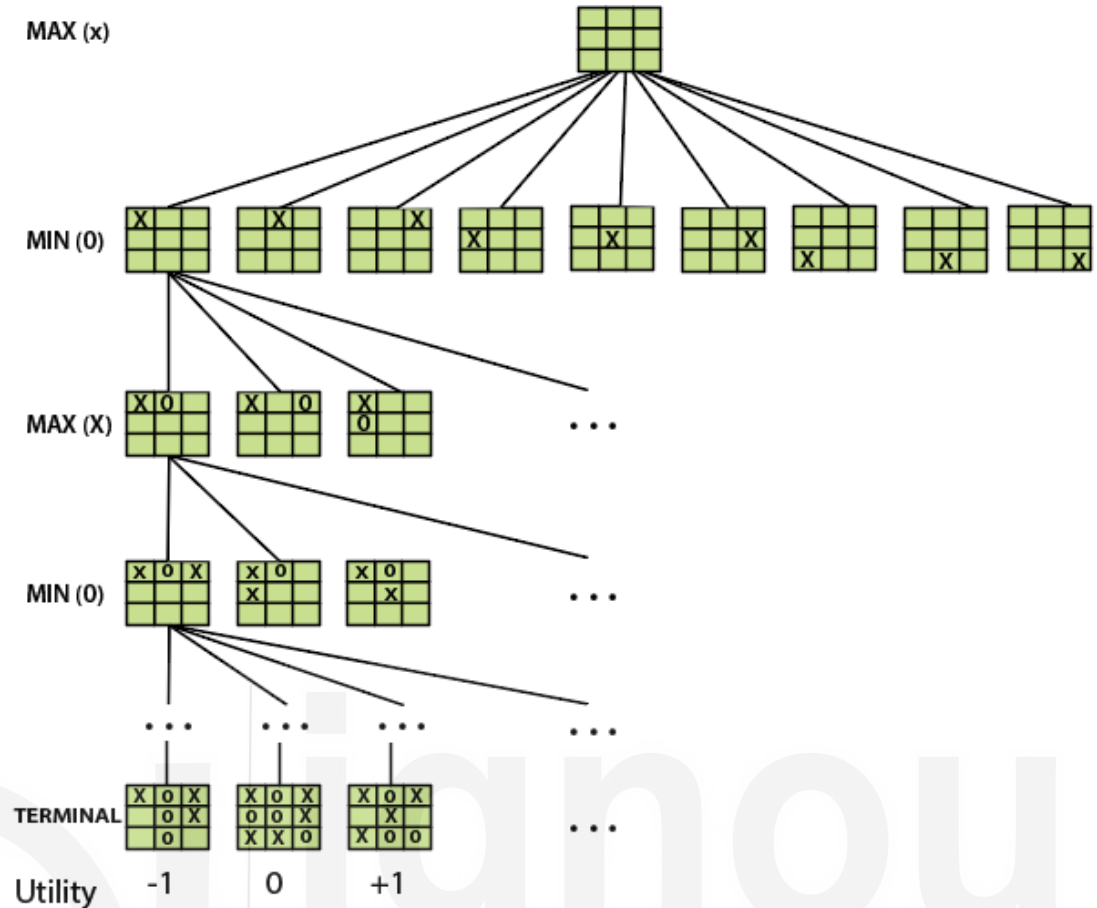


Fig. 17: A game-tree for tic-tac-toe

In a tic-tac-toe game playing, as shown in Figure 17, we have the following elements:

- **INITIAL STATE (S_0):** The top node in the game-tree represents the initial state in the tree and shows all the possible choices to pick out one.
- **PLAYER (s):** There are two players, **MAX** and **MIN**. **MAX** begins the game by picking the best move and placing **X** in the empty square box.
- **ACTIONS (s):** Both players can make moves in the empty boxes, chance by chance.
- **RESULT (s, a):** The moves made by **MIN** and **MAX** will decide the outcome of the game.
- **TERMINAL-TEST(s):** When all the empty boxes are filled, it will be the terminating state of the game.
- **UTILITY:** At the end, we will get to know who wins: **MAX** or **MIN**, and accordingly, the price will be given to them. If **MAX** wins, then the utility value is +1, if **MIN** wins, then the utility value is -1; and if **DRAW**, then the utility value is 0.

2.5.2 Issues in Adversarial Search

In a **normal search**, we follow a sequence of actions to reach the goal or to finish the game optimally. But in an **adversarial search**, the result depends on the players who will decide the result of the game. It is also obvious that the solution for the goal state will be an optimal solution because the player will try to win the game with the shortest path and within a limited time. **The Min-**

Max search algorithm is an example of adversarial search. **Alpha-beta Pruning** is used to reduce the search space.

Problem Solving Using
Search

2.6 MIN-MAX SEARCH STRATEGY

In artificial intelligence, minimax is a **decision-making** strategy under **game theory**, which is used to minimize the losing chances in a game and to maximize the winning chances. This strategy is also known as '**Min-Max**,' '**MM**,' or '**Saddle point**.' Basically, it is a two-player game strategy where *if one wins, the other loses the game*. This strategy simulates those games that we play in our day-to-day life. Like, if two people are playing chess, the result will be in favour of one player and against the other one. The person who will make his best try, *efforts as well as cleverness, will surely win*.

We can easily understand this strategy via a **game tree**, where nodes are the states of the game, and edges are moves that were made *by the players in the game*. Players will be two, namely:

- **MIN**: Decrease the chances of **MAX** winning the game.
- **MAX**: Increases his chances of winning the game.

They both play the game alternatively, i.e., turn by turn, and following the above strategy, i.e., if one wins, the other will definitely lose it. Both players look at one another as competitors and will try to defeat one another, giving their best.

In the minimax strategy, the result of the game or the utility value is generated by a **heuristic function** by propagating from the initial node to the root node. It follows the **backtracking technique** and backtracks to find the best choice. MAX will choose that path which will increase its utility value, and MIN will choose the opposite path, which could help it to minimize MAX's utility value.

2.6.1 MINIMAX Algorithm

The MINIMAX algorithm is a backtracking algorithm that backtracks to pick the best move out of several choices. MINIMAX strategy follows the **DFS (Depth-first search)** concept. Here, we have two players, **MIN** and **MAX**, and the game is played alternately between them, i.e., when **MAX** makes a move, then the next turn is of **MIN**. It means the move made by MAX is fixed, and he cannot change it. The same concept is followed in the DFS strategy, i.e., we follow the same path and cannot change in the middle. That's why in the MINIMAX algorithm, instead of BFS, we follow DFS. The following steps are used in the MINMAX algorithm:

- Generate the whole game tree, all the way down to the terminal states.
- Apply the utility function to each terminal state to get its value.
- Use the utility of the terminal states to determine the utility of the nodes one level higher up in the search tree.
- Continue backing up the values from the leaf nodes toward the root, one layer at a time.
- Eventually, the backed-up values reached the top of the tree; at that point, MAX chooses the move that leads to the highest value.

This is called a minimax decision because it maximizes the utility under the assumption that the opponent will play perfectly to minimize it. To better

understand the concept, consider the following game tree or search tree as shown in Figure 18.

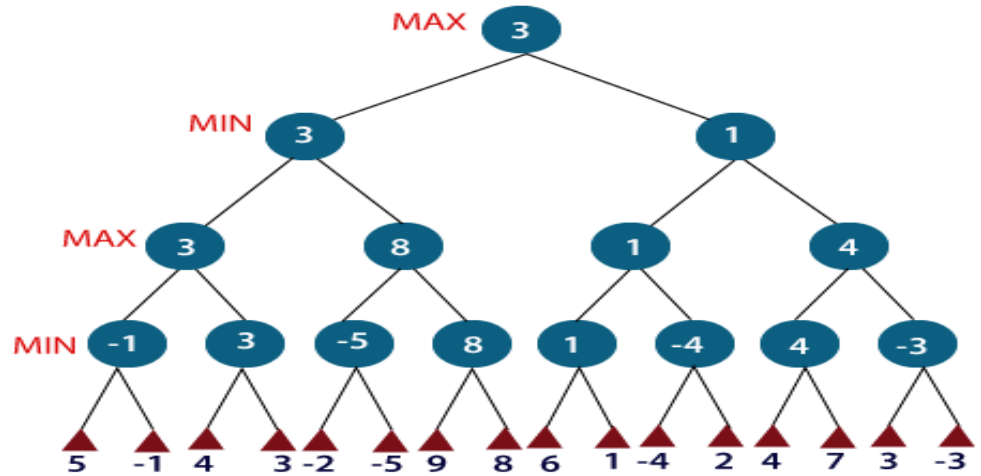


Fig 18 Two player game tree

In the figure above, the two players **MAX** and **MIN** are there. **MAX** starts the game by choosing one path and propagating all the nodes of that path. Now, **MAX** will backtrack to the initial node and choose the best path where his utility value will be the maximum. After this, it's **MIN** chance. **MIN** will also propagate through a path and again will backtrack, but **MIN** will choose the path that could minimize **MAX** winning chances or the utility value.

So, if the level is minimizing, the node will accept the minimum value from the successor nodes. If the level is maximizing, the node will accept the maximum value from the successor.

In other words, we can say that Minimax is a decision rule algorithm, which is represented as a game tree. It has applications in decision theory, game theory, statistics, and philosophy. Minimax is applied in two-player games. One is the MIN player, and the other is the MAX player. By agreement, the root of the game tree represents the MAX player. It is assumed that each player aims to do the best move for himself and therefore the worst move for his opponent in order to win the game. The question may arise, “How to deal with the contingency problem?” The answer is:

- Assuming that the opponent is rational and always optimizes its behaviour (opposite to us), we consider the best response. opponent's
- Then the minimax algorithm determines the best move

2.6.2 Working of Minimax Algorithm

Minimax is applicable for decision making for two agent systems participating in a competitive environment. These two players, P1 and P2, also known as the MIN and MAX players, maximize and minimize the utility value of the heuristic function. The algorithm uses recursion to search through the game tree and compute a minimax decision for the current state. We traverse the complete game tree in a depth-first search (DFS) manner to explore the node. MAX player always selects the maximum value, and MIN always selects the

minimum value from its successor's node. The initial value of MAX and MIN is set to $MAX = -\infty$ and $MIN = +\infty$. This is the worst value assigned initially, and as the algorithm progresses, these values are changed, and finally, we get the optimal value.

Example 1: Let's take an example of a two-player game tree search (shown in Figure 19a) to understand the working of the Minimax algorithm.

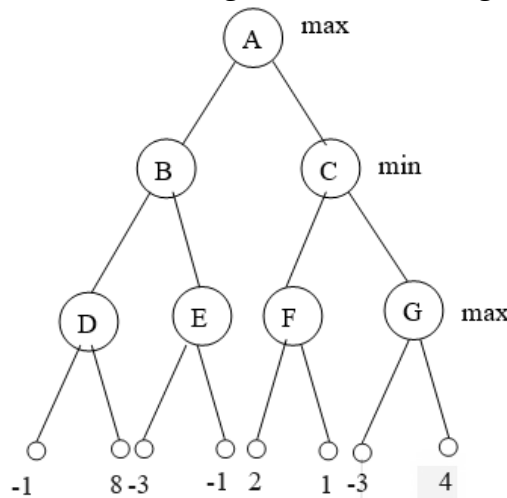


Fig 19a Two player game tree

The initial value of MAX and MIN is set to $MAX = -\infty$ and $MIN = +\infty$. The tree is traversed in a DFS manner. So, we start from node A, then move to node B, and then D.

Now at node D [$MAX = -\infty$]. Now, at D, it first checks the left child (which is a terminal node) with value -1. This node returns a value of $MAX = (-\infty, -1) = -1$. So, the modified value at node D is [$MAX = -1$]. Next, we proceed to the right child of Node D (which has a terminal value of 8) and compare this value (8) with the previous value at node D. That is $MAX = \max(-1, 8) = 8$. So, the final value at node D is 8.

Similarly, the value at node E (which is a Max node) is $MAX = \max(-\infty, -3) = -3$, then $\max(-3, 2) = 2$.

So, at node B, which is at MIN level, select the minimum value from its successor node D and E as $MIN = \min(8, 2) = 2$.

Similarly, the value at node F (which is also Max node) is $MAX = \max(-\infty, 1) = 1$, then $\max(1, -3) = 1$, and

The value at node G (which is also MAX node) is $MAX = \max(-\infty, -3) = -3$, and then $\max(-3, 4) = 4$.

Thus, at node C, which is also at MIN level, select the minimum value from its successor node F and G as $MIN = \min(1, 4) = 1$.

Now, the value at node B and C is 2 and 1 respectively.

Thus, finally, the value at node A, which is at MAX level, is

$MAX = \max(2, 1) = 2$.

The final game tree with max or min value at each node and optimal path, with shaded line $A \rightarrow C \rightarrow F \rightarrow 1$, is shown in the following figure 19(b).

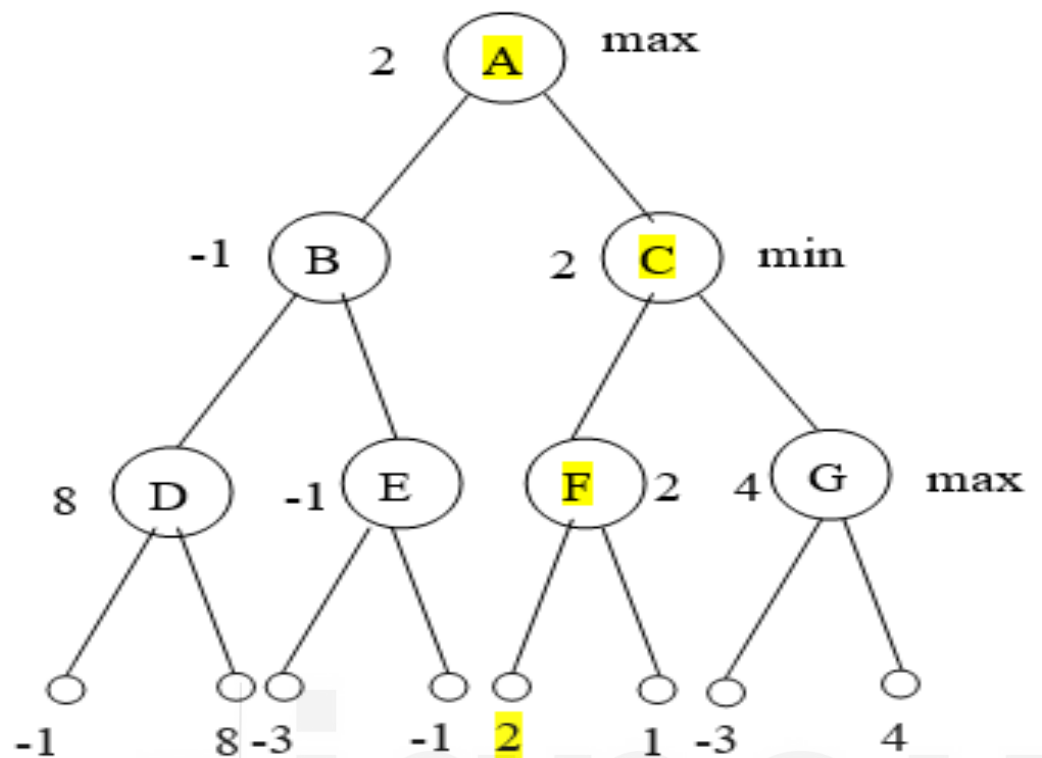
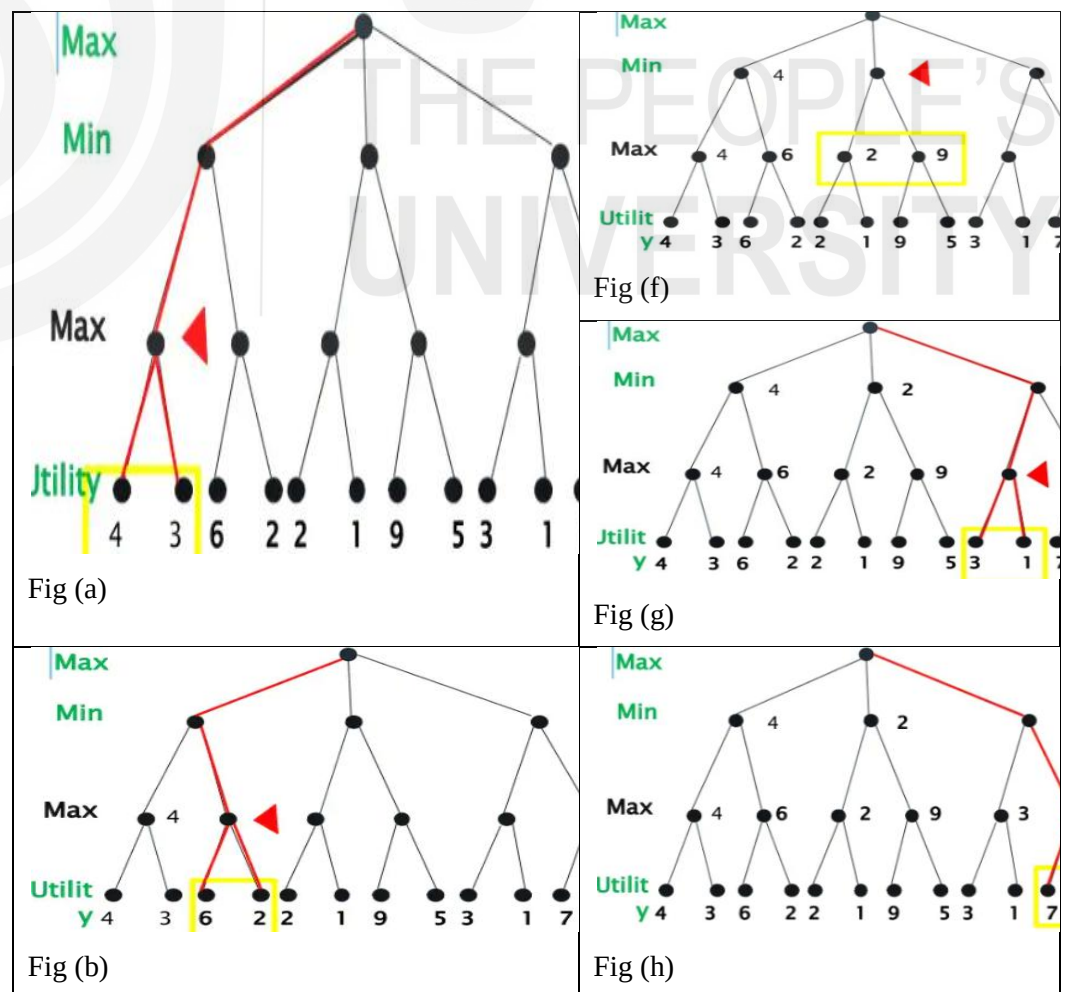
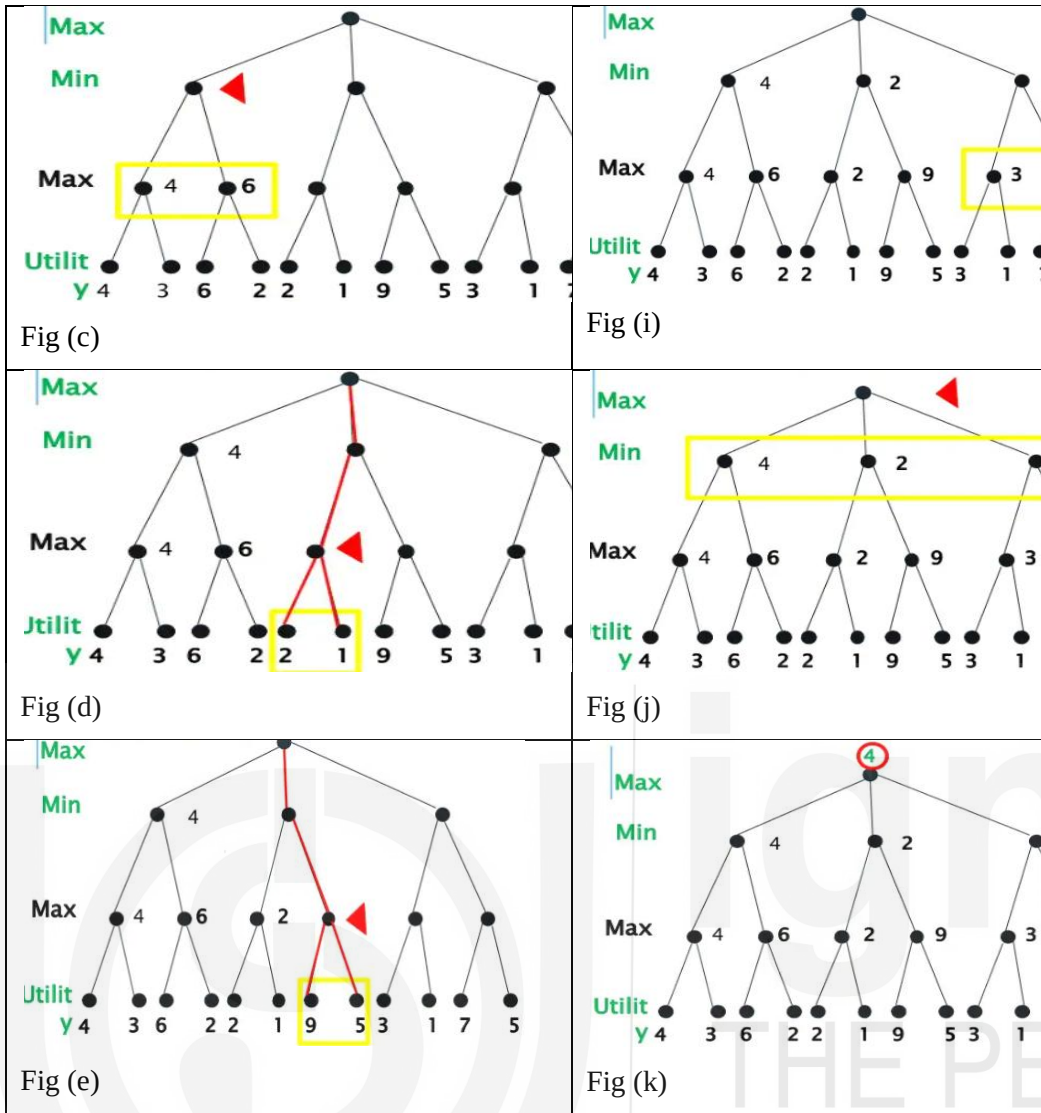


Fig 19(b) Game tree with final value at each node with optimal path

Example 2: Consider the following two-player game tree search. The working of Minimax algorithm is illustrated from fig (a)-fig(k)





2.6.3 Properties of Minimax Algorithm:

1. **Complete:** The Minimax algorithm is complete if the tree is finite.
2. **Optimal:** If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be an optimal solution. Minimax is Optimal.
3. **Time complexity:** $O(b^m)$, where b : Branching Factor and m is the maximum depth of the game tree.
4. **Space complexity:** $O(bm)$

For example, in a chess playing game, $b = 35$, $m \approx 100$ for “reasonable” games. In this case, the exact solution is completely infeasible

2.6.4 Advantages and disadvantages of Minimax search

Advantages:

- Returns an optimal action, assuming perfect opponent play.
- Minimax is the simplest possible (reasonable) game search algorithm.

Disadvantages:

- It's completely infeasible in practice.

- When the search tree is too large, we need to limit the search depth and apply an evaluation function to the cut-off states.

2.7 ALPHA-BETA PRUNING

The drawback of the Minimax strategy is that it explores each node in the tree deeply to provide the best path among all the paths. This increases its time complexity. If b is the branching factor and d is the depth of the tree, then the time complexity of the MINIMAX algorithm is $O(b^d)$, that is exponential. But as we know, the performance measure is the first consideration for any optimal algorithm. Alpha-beta pruning is a method to reduce (prune) search space. Using Alpha-Beta pruning, the Minimax algorithm is modified. Therefore, alpha-beta pruning reduces this drawback of the minimax strategy by exploring the nodes of the search tree.

The method used in alpha-beta pruning is that its **cut-off the search** by exploring a smaller number of nodes. It makes the same moves as a minimax algorithm does, but it prunes the unwanted branches using the pruning technique (discussed in adversarial search). Alpha-beta pruning works on two threshold values, i.e., α (**alpha**) and β (**beta**).

- α : It is the highest value a **MAX** player can have. The initial value of α is set to a negative infinity value, that is $\alpha = -\infty$. As the algorithm progresses, its value may change and finally get the best (highest) value.
- β : It is the best lowest value a **MIN** player can have. The initial value of β is set to a positive infinity value, that is $\beta = +\infty$. As the algorithm progresses, its value may change and finally get the best (lowest) value.

So, each MAX node has α -value, which never decreases, and each MIN node has β -value, which never increases. The main condition required for alpha-beta pruning is $\alpha \geq \beta$, that is if $\alpha \geq \beta$, then prune (cut) the branches; otherwise, proceed.

Note: The alpha-beta pruning technique can be applied to trees of any depth, and it is possible to prune the entire sub-trees easily.

2.7.1 Working of Alpha-beta Pruning

As we know, there are two-parameter is defined for Alpha-beta pruning, namely alpha (α) and beta (β). The initial value of alpha and beta is set to $\alpha = -\infty$ and $\beta = +\infty$. As the algorithm progresses, its values are changed accordingly. Note that in Alpha-beta pruning (cut), at any node in a tree, if $\alpha \geq \beta$, then prune (cut) the next branch else search is continued. Note the following point for alpha-beta pruning:

- The MAX player will only update the value of α (on MAX level).
- The MIN player will only update the value of β (on MIN level).
- We will only pass the α and β value from top to bottom (that is, from any parent to child node, but never from child to parent node).
- While backtracking the tree, the node values will be passed to the upper node instead of the values of α and β .
- Before going to the next branch of the node in a tree, we check the value of α and β . If the value of $\alpha \geq \beta$, then prune (cut) the next

(unnecessary) branches (i.e., no need to search the remaining branches where the condition $\alpha \geq \beta$ is satisfied) else search continued.

Consider the example below of a game tree where **P** and **Q** are two players. The game will be played alternately, i.e., chance by chance. Let **P** be the player who will try to win the game by maximizing their winning chances. **Q** is the player who will try to minimize **P**'s winning chances. Here, α will represent the maximum value of the nodes, which will be the value for **P** as well. β will represent the minimum value of the nodes, which will be the value of **Q**.

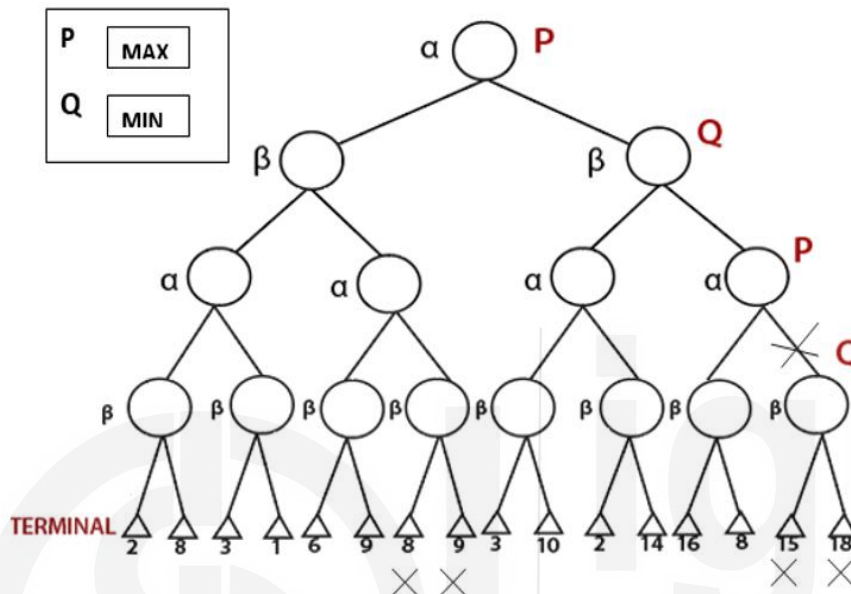


Fig 20: Alpha-beta pruning

- Any one player will start the game. Following the DFS order, the player will choose one path and will reach its depth, i.e., where he will find the **TERMINAL** value.
- If the game is started by player P, he will choose the maximum value in order to increase its winning chances with maximum utility value.
- If the game is started by player Q, he will choose the minimum value in order to decrease the winning chances of A with the best possible minimum utility value.
- Both will play the game alternately.
- The game will be started from the last level of the game tree, and the value will be chosen accordingly.
- Like in Figure 5, the game is started by player Q. He will pick the leftmost value of the **TERMINAL** and fix it for beta (β). Now, the next **TERMINAL** value will be compared with the β -value. If the value is smaller than or equal to the β -value, replace it with the current β -value otherwise no need to replace the value.
- After completing one part, move the achieved β -value to its upper node and fix it for the other threshold value, i.e., α .
- Now, it's P's turn; he will pick the best maximum value. P will move to explore the next part only after comparing the values with the current ones α -value. If the value is equal to or greater than the current α -value, then only it will be replaced; otherwise, we will prune the values.
- The steps will be repeated until the desired result is obtained.
- So, the number of pruned nodes in the above example is **four**, and MAX wins the game with the maximum **UTILITY** value, i.e., **3**.

The rule which will be followed is: “**Explore nodes, if necessary, otherwise prune the unnecessary nodes.**”

Note: It is obvious that the result will have the same **UTILITY** value that we may get from the MINIMAX strategy.

Alpha beta cut-off (or pruning):

1. for each node store limit $[\alpha, \beta]$.

2. Update $[\alpha, \beta]$,

where α is the lower bound at max node; it can't decrease.

β is the upper bound at min node; it can't increase.

3. If α value of a max node is greater than β value of its parent ($\alpha \geq \beta$), the subtree of that max node need not be evaluated (i.e., pruned).

4. If β value of a min node is lesser than α value of its parent ($\beta \leq \alpha$), the subtree of that min node need not be evaluated (i.e., pruned).

Example 1: Let's take an example of two-player search tree (Figure 21) to understand the working of alpha-beta pruning.

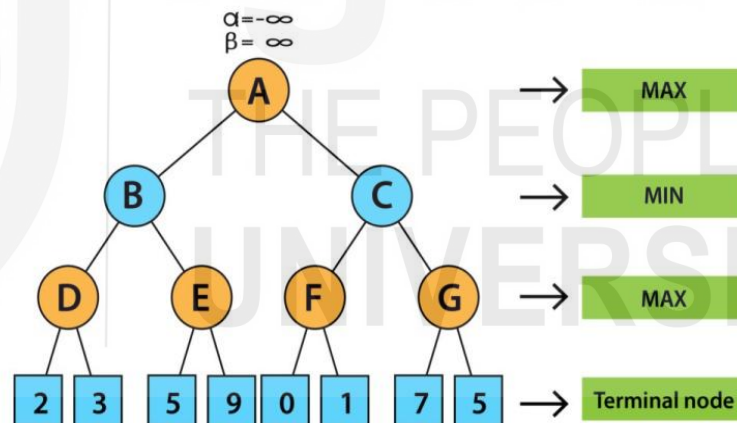


Fig. 21: Two player search tree

We initially start the search by setting the initial value of $\alpha = -\infty$ and $\beta = +\infty$ to root **node A**.

Note the following important point to apply the Alpha-beta pruning:

- We will only pass the α and β value from top to bottom (that is from any parent to child node), but never from child to parent node.
- While backtracking the tree (from bottom to top node), the node values will be passed to upper node instead of values of α and β .
- Before exploring the next branch in a tree, we check $\alpha \geq \beta$. If **YES**, then prune (cut) the next (unnecessary) branches (i.e., no need to search

the remaining branches where the condition $\alpha \geq \beta$ is satisfied) else search continued.

- The MAX player will only update the value of α (on MAX level) and the MIN player will only update the value of β (on MIN level).

Step 1: We traverse the tree in a depth-first search (DFS) manner and assign (pass) this value of α and β down to subsequent nodes **B** and then to node **D** as $[\alpha = -\infty; \beta = +\infty]$.

Now at node D $[\alpha = -\infty, \beta = +\infty]$, since node D is at MAX level, so only α value will be changed. Now, at D, it first checks the left child (which is a terminal node) with value 2. This node returns a value of 2. Now, the value of α at node D is calculated as $\alpha = \max(-\infty, 2) = 2$. So the modified value at node D is $[\alpha = 2, \beta = +\infty]$. To decide whether it's worth looking at its right node or not, we check $\alpha \geq \beta$. The answer is NO since $2 \not\geq +\infty$. So, proceed and search continues for the right child of Node D.

The value of the right child (terminal node with value=3) of D returns a value 3. Now at D, the value of α is compared with the terminal node value 3, that is, $\alpha = \max(2, 3) = 3$. Now the value of Node(D)=3, and the final values of α and β is updated at node D as $[\alpha = 3, \beta = +\infty]$ as shown in figure 21(a).

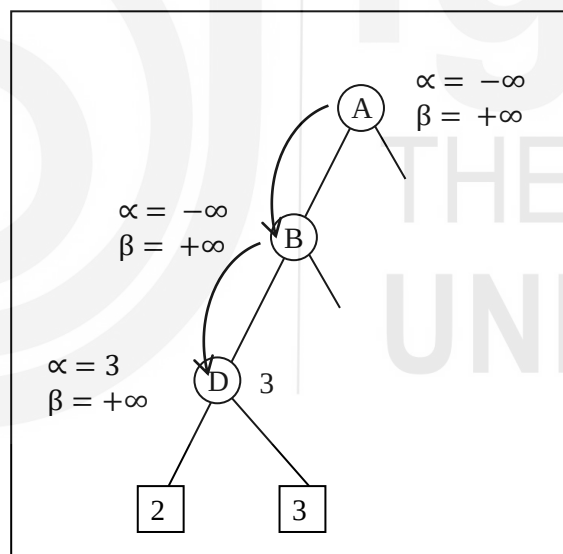


Fig 21(a)

Step 2: We backtrack from node D to B. Note that, while backtracking the node in a tree, the node values of D(=3) will be passed to the upper node B instead of the values of α and β . Now the value of node(B) = node(D) = 3. Since B is at MIN level, so only β value will be changed. Now at node **B** $[\alpha = -\infty, \beta = 3]$ (note that β is changed from $+\infty$ to 3). Here we again check $\alpha \geq \beta$. It is False, so search is continued on right side of B, as shown in Figure 21(b).

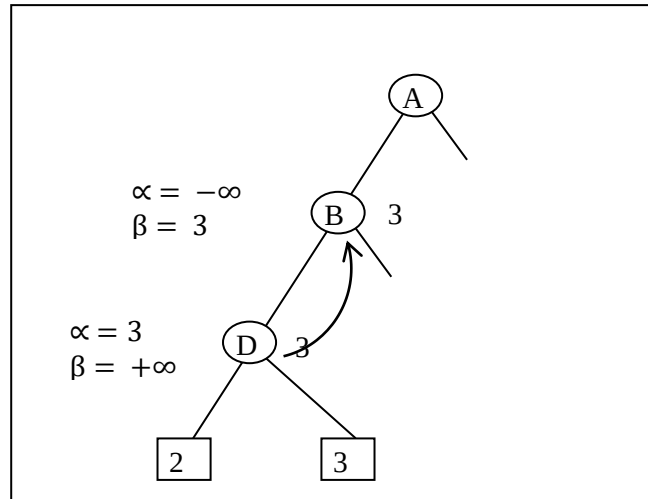


Fig. 21(b)

Step 3: B now calls E, and we pass the α and β value from top node B to bottom node E as $[\alpha = -\infty, \beta = 3]$. Since node E is at MAX level, so only α value will be changed. Now, at E, it first checks the left child (which is a terminal node) with value 5. This node returns a value of 5.

Now, the value of α at node E is calculated as $\alpha = \max(-\infty, 5) = 5$, so value of Node(E)=5 and modified value of α and β at node E is $[\alpha = 5, \beta = 3]$. To decide whether it's worth looking at its right node or not, we check $\alpha \geq \beta$. The answer is **YES**, since $5 \geq 3$. So, we prune (cut) the right branch of E, as shown in Figure 21(c).

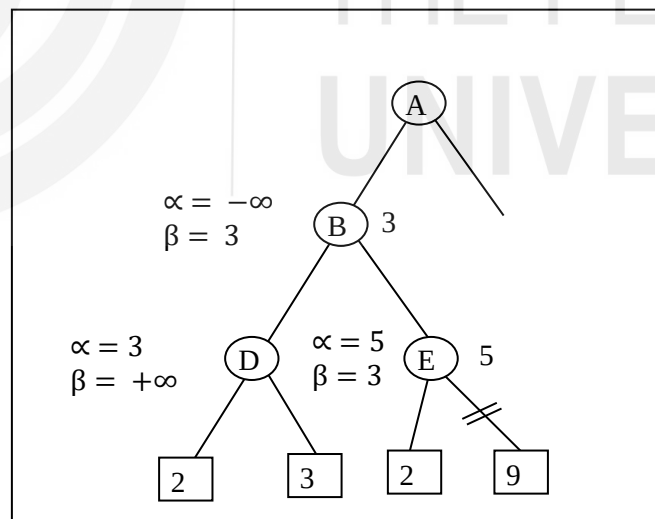


Fig. 21(c)

Step 4: We backtrack from node E to B. Note that, while backtracking, the node values of E(=5) will be passed to the upper node B, instead of the values of α and β . E returns a value 5 to B. Since B is at MIN level, so only β value will be changed. Previously, at node B $[\alpha = -\infty, \beta = 3]$, but now $\beta = \min(3, 5) = 3$, so, there is no change in β value and value of node(B) is still 3. Thus, finally, the modified value at node B is $[\alpha = -\infty, \beta = 3]$.

We backtrack from node B to A. Again, note that, while backtracking the tree, the value of node(B)=3 will be passed to the upper node A, instead of the values of α and β . **Now value of Node(A)=3.**

Since A is at MAX level, so only α value will be changed. Previously, at node A [$\alpha = -\infty, \beta = +\infty$] and after comparing value of node(B)=3 with old value of α at node A, that is $\alpha = \max(-\infty, 3) = 3$. Thus, finally, at node A [$\alpha = 3, \beta = +\infty$] and value of Node(A)=3. we check $\alpha \geq \beta$, it is False, so proceed on the right side. Now, we have completed the left sub-tree of A and proceed towards the right subtree, as shown in Figure 21(d).

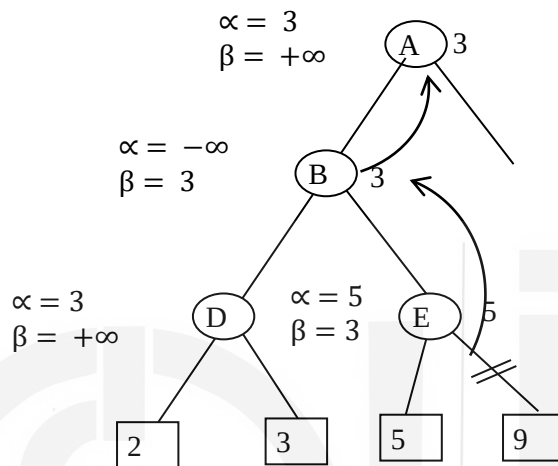


Fig. 21(d)

Step 5. Now at node C, we pass the α and β value from top node A to bottom node C as [$\alpha = 3, \beta = +\infty$]. **Check,** $\alpha \geq \beta$. The answer is NO. So, the search is continued. Now pass the α and β value from top node C to bottom node F as [$\alpha = 3, \beta = +\infty$]. Since F is at MAX level, so only α value will be changed. Now, at F, it first checks the left child (which is a terminal node) with value 0. This node returns a value of 0. Now, the value of α at node F is calculated as $\alpha = \max(3, 0) = 3$. So the modified value at node F is [$\alpha = 3, \beta = +\infty$]. To decide whether it's worth looking at its right node or not, we check $\alpha \geq \beta$. The answer is NO since $3 \not\geq +\infty$. So, proceed and search continues for the right child of Node F.

The value of the right child (terminal node with value = 1) of F returns a value 1, so finally, the value of node(F)=1. Now at F, the value of α is compared with the terminal node value 1, that is, $\alpha = \max(3, 1) = 3$, and the final values of α and β is updated at node F as [$\alpha = 3, \beta = +\infty$] as shown in figure 21(e).

Step 6. **We** backtrack from node F to C. Note that, while backtracking the tree, the node values of F(=3) will be passed to the upper node C. Now the value of **node(C) = node(F) = 1.**

Since C is at MIN level, so only β value will be changed. Previously, at node C [$\alpha = 3, \beta = +\infty$]. Now, the old value of $\beta = +\infty$ is compared with value of node(F) = node(C) = 1. That is, $\beta = \min(+\infty, 1) = 1$. Thus, finally, at node B [$\alpha = 3, \beta = 1$]. **Now we check,** $\alpha \geq \beta$. It is TRUE, so we prune (cut) the right branch of node C. That is, node G will be pruned, and the algorithm stops searching on the right subtree of node C.

Thus, finally, **we** backtrack from node C to A, and node C returns the value 1 to node A. Since A is a MAX node, only α value will be changed.

Previously, at node A [$\alpha = 3, \beta = +\infty$] and after comparing value of node(C)=1 with old value of α at node A, that is $\alpha = \max(3, 1) = 3$. Thus, finally, at node A [$\alpha = 3, \beta = +\infty$] and value of Node(A)=3. Now, we have completed the right subtree of A also.

Following is the final game tree, showing the nodes that are computed and nodes that are pruned (cut) during the search process of Alpha-beta pruning. Here, the optimal value for the maximizer is 3, and there are 3 terminal nodes that are pruned (9, 7, and 5). The optimal search path is $A \rightarrow B \rightarrow D \rightarrow 3$.

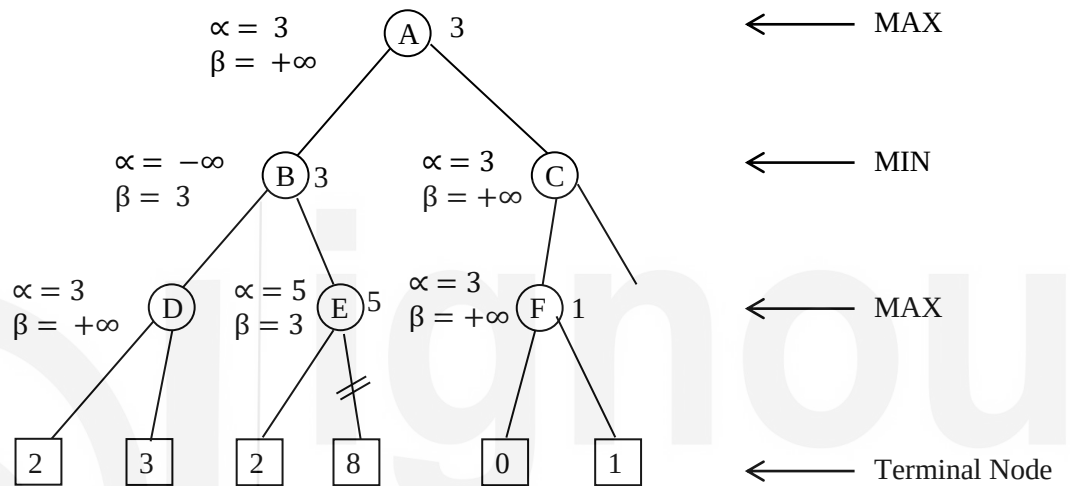


Fig. 21(e)

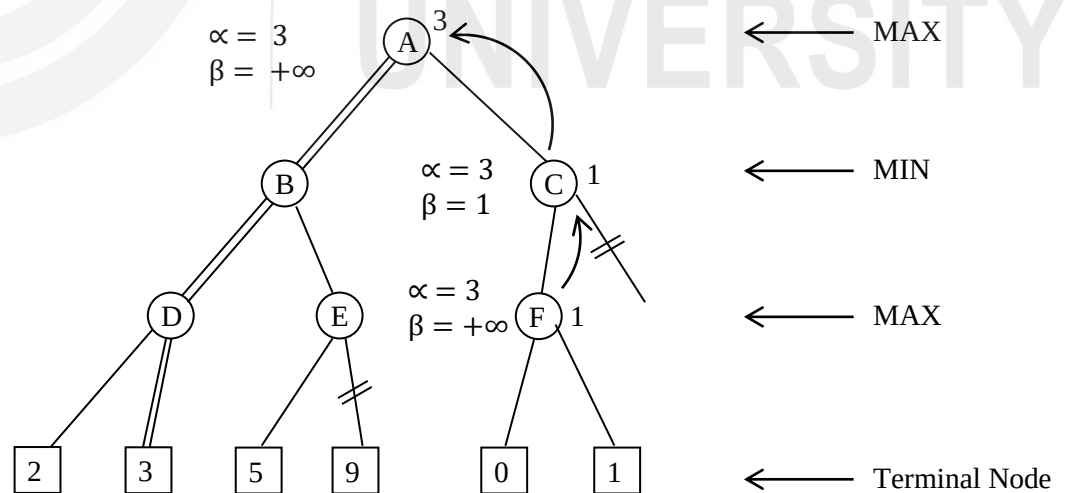


Fig. 21(f)

Example 2: Consider the following game tree (Figure 22) in which the root is the maximizing node and children are visited from left to right. Find which nodes are pruned by the Alpha-beta pruning.

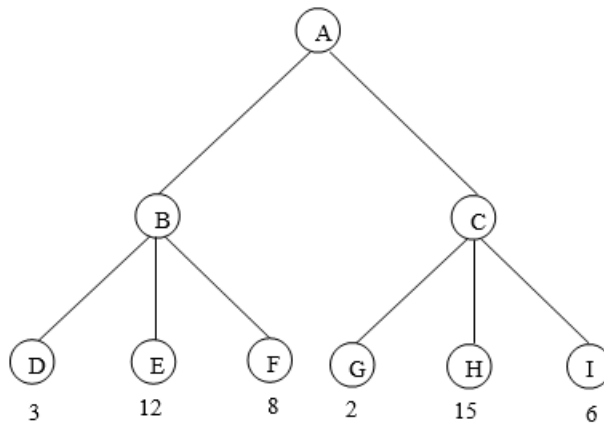


Fig. 22: Two Player search tree

Solution:

Step 1: We start the search by setting the initial value for node A as $[\alpha = -\infty, \beta = +\infty]$. We traverse the node in a depth-first search (DFS) manner, so we assign the same value of α and β to node B $[\alpha = -\infty, \beta = +\infty]$. Since node B is at MIN level, so only β value will be changed at node B. Now, at D, it looks at its left child (terminal node), which returns a value 3 to D. So, we compare the old value of β at node B with this terminal node value 3, that is $\beta = \min(+\infty, 3) = 3$. So modified value of α and β to node B is $[\alpha = -\infty, \beta = 3]$.

To decide whether it is worth looking at the right subtree of B, we check $\alpha \geq \beta$. The answer is NO, since $-\infty \not\geq 3$. So, proceed and search continues for the right child of Node B, that is E.

The terminal value of E = 12. Now, the value of right child terminal E(=12) is compared with the previous old value of $\beta = 3$, that is, $\beta = \min(3, 12) = 3$. So no change in β value. So, at present, the current modified value of α and β at node B is the same $[\alpha = -\infty, \beta = 3]$. Again check, $\alpha \geq \beta$. The answer is NO. So, proceed and search continues for the right child of Node B, that is, F. The terminal value of F=8. The value of the right child terminal at F(=8) is compared with the previous old value of $\beta = 3$, that is, $\beta = \min(8, 3) = 3$. So no change in β value. So, finally, the value of **Node(B)=3** and the modified value of α and β at node B $[\alpha = -\infty, \beta = 3]$ as shown in Figure 22(a)

Step 2: We backtrack from node B to A. Note that, while backtracking the tree, the node values of B(=3) will be passed to the upper node A, instead of the values of α and β . **Now the value of node(A) = node(B) = 3.** Since A is at the MAX level, so only α value will be changed. Previously, at node A $[\alpha = -\infty, \beta = +\infty]$ and after comparing value of node(B)=3 with old value of α at node A, that is $\alpha = \max(-\infty, 3) = 3$. Thus, finally, at node A $[\alpha = 3, \beta = +\infty]$ and value of **Node(A)=3**.

To decide whether it's worth looking at the right node of A or not, we check $\alpha \geq \beta$. The answer is NO since $3 \not\geq +\infty$. So, proceed and search continues for the right child of Node A. Now, we have completed the left sub-tree of A and proceed towards the right subtree.

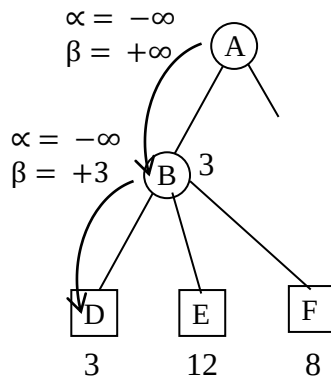


Fig22(a)

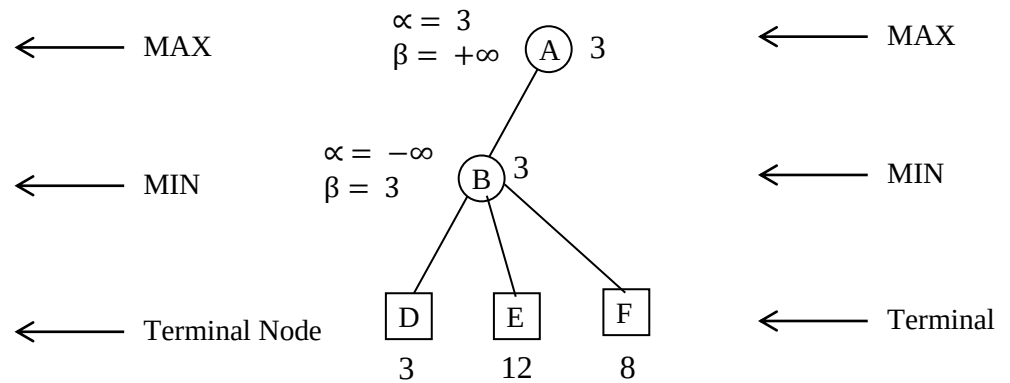


Fig 22(b)

Step 3: Now, at node C, we pass the α and β value from top node A to bottom node C as $[\alpha = 3, \beta = +\infty]$. **Check,** $\alpha \geq \beta$. The answer is NO. So, continue the search on the right side. Since C is at MIN level, so only β value will be changed.

Now, we first check the left child (terminal) of node C, that is, G=2. So, we compare the old value of β at node C with this terminal node value 2, that is $\beta = \min(+\infty, 2) = 2$. So value of Node(C)=2 and modified value of α and β at node C is $[\alpha = 3, \beta = 2]$. Now, before proceeding to the next, we again check $\alpha \geq \beta$. The answer is YES. So, we prune (cut) the right branch of node C. That is **node H, and I** will be pruned (cut), and the algorithm stops searching on the right subtree of node C, as shown in Figure 22(c).

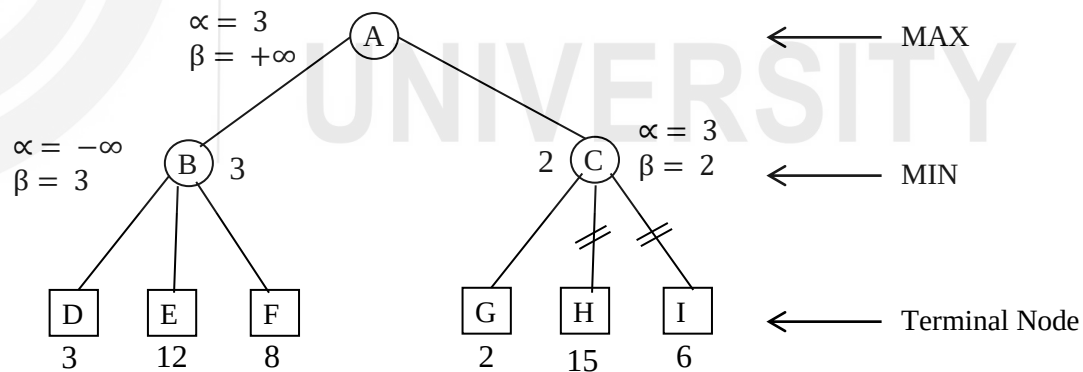


Fig 22(c)

Step 4: Finally, we backtrack from node C to A. Note that, while backtracking the node in a tree, the node values of C(=2) will be passed to the upper node A, instead of the values of α and β . The previous node(A)=3 value is compared with this new node(C)=2 value. The best value at node(A)= $\alpha = \max(3, 2) = 3$.

The previous α and β value at node A $[\alpha = 3, \beta = +\infty]$. Since A is at MAX level, only α value is change. So, we compare old $\alpha = 3$ value with value at node(C)=2. That is $\alpha = \max(3, 2) = 3$. Thus, there is no change in α value as well.

Thus finally, α and β value at node A is $[\alpha = 3, \beta = +\infty]$ and value of Node(A)=3. So, the optimal value for the maximizer is 3, and there are 2 terminal nodes that are pruned (H and I). The optimal search path is $A \rightarrow B \rightarrow D$ (as shown in Figure 22(d)).

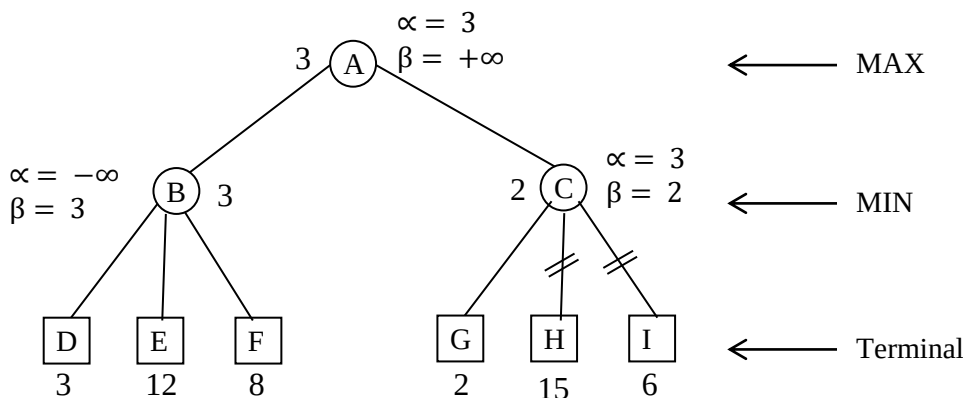


Fig 22(d)

2.7.2 Move ordering of Alpha-beta pruning

The effectiveness of Alpha-beta pruning is highly dependent on the order in which each node is examined. Move order is an important aspect of alpha-beta pruning. We have two types of move ordering:

Worst-case ordering: In some cases, the alpha-beta pruning algorithm does not prune any of the leaves of the tree and works exactly as the MiniMax algorithm. In this case, it consumes more time because of alpha-beta factors; such a move of pruning is called a worst ordering. The time complexity for such an order is $O(b^m)$ where b: Branching Factor and m is the depth of the tree.

Best (ideal) case ordering: The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and the best move occurs at the left side of the tree. We apply DFS, hence it first searches left of the tree and goes deep twice as the minimax algorithm does in the same amount of time. The time complexity for the best-case order is $O(b^{m/2})$ (since we search only the left sub-tree, not the right subtree).

Note that pruning does not affect the final result. Good move ordering improves the effectiveness of pruning. With ideal case ordering, the time complexity is $O(b^{\frac{m}{2}})$

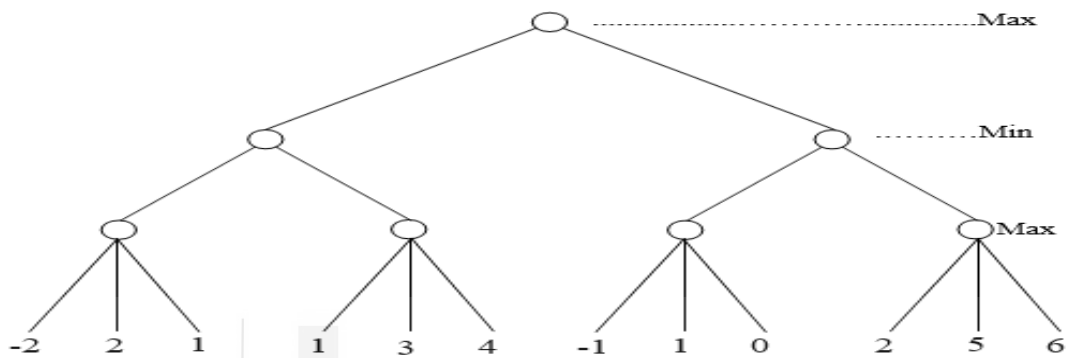
In Alpha-beta pruning:

- α value can never decrease and β value can never increase. Search can be discontinued at a node if:
- It is a Max node and $\alpha \geq \beta$ it is beta cutoff
- It is a Min node and $\beta \leq \alpha$ it is alpha cutoff.

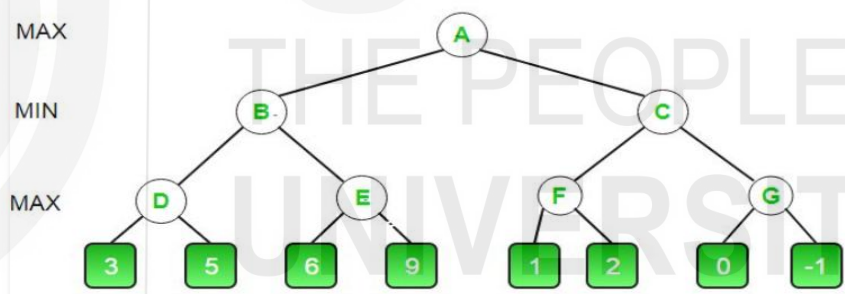
Check Your Progress 2

Q1. Compare the MINIMAX and Alpha-Beta Pruning algorithm with respect to Time complexity.

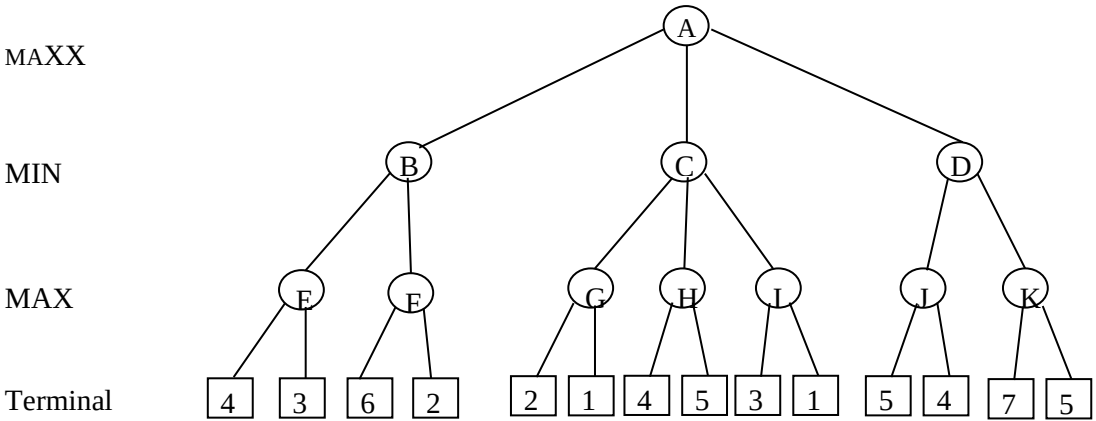
Q2. Consider the following Minimax game tree search in which the root is the maximizing node, and the children are visited from left to right. Find the value of the root node of the game tree?



Q3. Apply Alpha-Beta pruning algorithm on the following graph and find which node(s) are pruned?



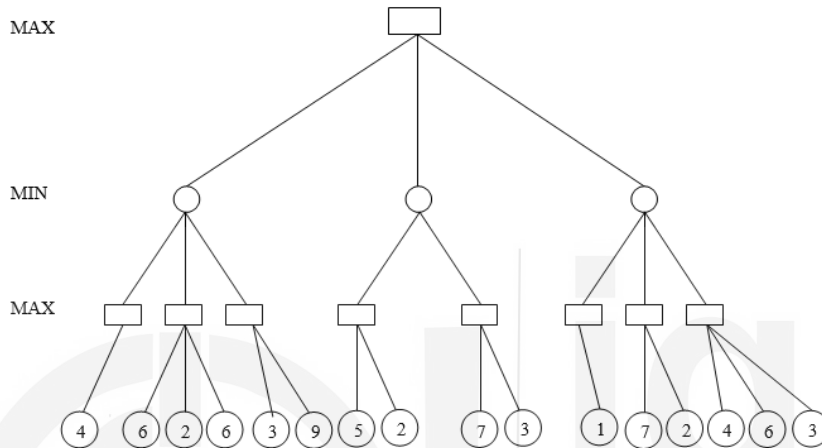
Q4. Consider the following Minimax game tree search in which the root is the maximizing node, and children are visited from left to right.



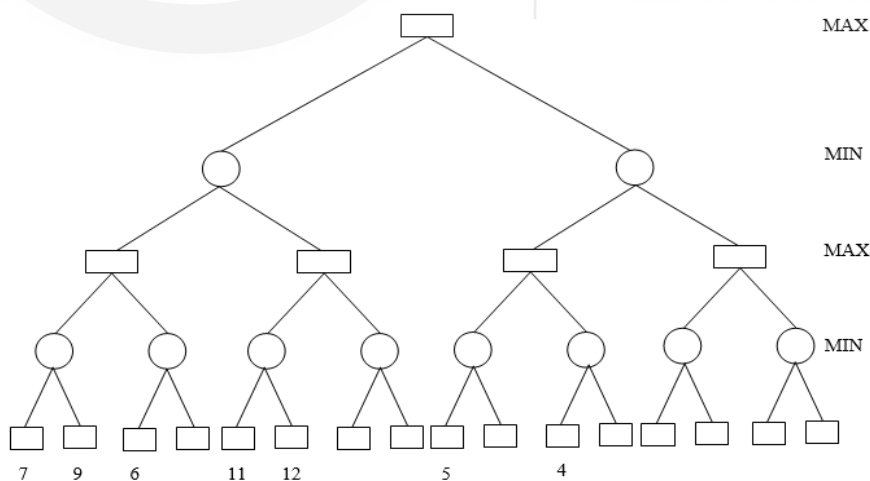
- (a) Find the value of the root node of the game tree?
 (b) Find all the nodes pruned in the tree?
 (c) Find the optimal path for the maximizer in a tree?

Problem Solving Using
Search

Q5. Consider the following Minimax game tree search in which the root is maximizing node, and the children are visited from left to right. Find what will be the value propagated at the root?

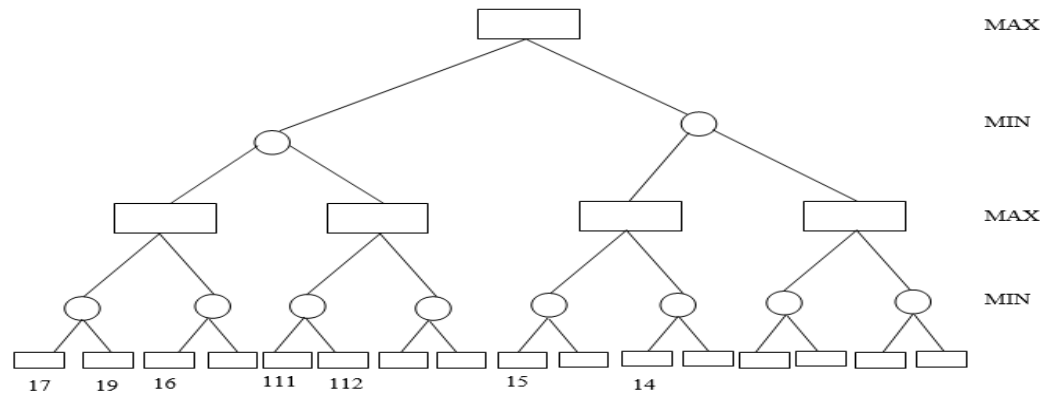


Q6. Consider the following Minimax game tree search in which root is maximizing node and children are visited from left to right. Find the value of the root node of the game tree?



Multiple Choice Question

Q7. Consider the following Minimax game tree search in which root is maximizing node and children are visited from left to right. Find the value of the root node of the game tree?



A. 14

B. 17

C. 111

D. 112

2.8 SUMMARY

- Before an AI problem can be solved, it must be represented in a **state space**. Among all possible states, there are two special states called the **initial state** (the start point) and the **final state** (the goal state).
- A **successor function (a set of operators)** is used to change the state. It is used to move from one state to another.
- A state space is a set of all possible states of a problem.
- A **state space** essentially consists of a set of nodes representing each state of the problem, arcs between nodes representing the legal moves from one state to another, an initial state, and a goal state. Each state space takes the form of a tree or a graph.
- The process of searching means a sequence of actions that takes you from an initial state to a goal state.
- Search is fundamental to the problem-solving process. Search means the problem is solved by using the rules, in combination with an appropriate **control strategy**, to move through the problem space until a path from an **initial state** to a **goal state** is found.
- A **problem space** is represented by a directed graph, where *nodes* represent *search states*, and *paths* represent the *operators* applied to change the state.
- In general, a state space is represented by 4 tuples as follows: $S_s: [S, s_0, O, G]$, Where **S**: Set of all possible states (possibly infinite), **s_0** : start state (initial configuration) of the problem, $s_0 \in S$. **O**: Set of production rules (or a set of state transition operators) used to change the state from one state to another. It is the set of arcs (or links) between nodes.
- Adversarial search is a game-playing technique where the agents are surrounded by a competitive environment. A conflicting goal is given

to the agents (multi-agent). These agents compete with one another and try to defeat one another in order to win the game. Such conflicting goals give rise to adversarial search.

- In a **normal search**, we follow a sequence of actions to reach the goal or to finish the game optimally. However in an **adversarial search**, the result depends on the players who will decide the result of the game. It is also obvious that the solution for the goal state will be an optimal solution because the player will try to win the game with the shortest path and under limited time.
- There are 2 types of adversarial search: **Minimax Algorithm** and **Alpha-beta Pruning**.
- Minimax is a two-player (namely MAX and MIN) game strategy where *if one wins, the other loses the game*. This strategy simulates those games that we play in our day-to-day life. Like, if two people are playing chess, the result will be in favour of one player and will go against the other one. **MIN**: Decreases the chances of **MAX** winning the game, and **MAX**: Increases his chances of winning the game. They both play the game alternatively, i.e., turn by turn, and following the above strategy, i.e., if one wins, the other will definitely lose it. Both players look at one another as competitors and will try to defeat one another, giving their best.
- In the minimax strategy, the result of the game or the utility value is generated by a **heuristic function** by propagating from the initial node to the root node. It follows the **backtracking technique** and backtracks to find the best choice. MAX will choose that path which will increase its utility value, and MIN will choose the opposite path, which could help it to minimize MAX's utility value.
- The drawback of minimax strategy is that it explores each node in the tree deeply to provide the best path among all the paths. This increases its time complexity.
- If b is the branching factor and d is the depth of the tree, then the time complexity of the MINIMAX algorithm is $O(b^d)$ that is exponential.
- Alpha-beta pruning is an advanced version of the MINIMAX algorithm. Therefore, alpha-beta pruning reduces the drawback of the minimax strategy by exploring the nodes of the search tree.
- The alpha-beta pruning method **cuts off the search** by exploring a smaller number of nodes. It makes the same moves as a minimax algorithm does, but it prunes the unwanted branches using the pruning technique.
- Alpha-beta pruning works on two threshold values, i.e., α (**alpha**) and β (**beta**). α : It is the highest value a **MAX** player can have. The initial value of α is set to a negative infinity value, that is $\alpha = -\infty$. As the algorithm progresses, its value may change and finally get the best (highest) value. β : It is the best lowest value a **MIN** player can have. The initial value of β is set to a positive infinity value, that

is $\beta = +\infty$. As the algorithm progresses, its value may change and finally get the best (lowest) value.

- So, each MAX node has α value, which never decreases, and each MIN node has β value, which never increases. The main condition required for alpha-beta pruning is $\alpha \geq \beta$, that is if $\alpha \geq \beta$, then prune (cut) the branches; otherwise, the search continues.
- As we know, there are two-parameter is defined for Alpha-beta pruning, namely alpha (α) and beta (β). The initial value of alpha and beta is set to $\alpha = -\infty$ and $\beta = +\infty$. As the algorithm progresses, its values are changed accordingly. Note that in Alpha-beta pruning (cut), at any node in a tree, if $\alpha \geq \beta$, then prune (cut) the next branch else search is continued.
- The effectiveness of Alpha-beta pruning is highly dependent on the order in which each node is examined.
- **Worst-case ordering:** In some cases, the alpha-beta pruning algorithm does not prune any of the leaves of the tree and works exactly as the MiniMax algorithm. In this case, the time complexity is $O(b^m)$ where b is the branching Factor and m is the depth of the tree.
- **Best (ideal) case ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and the best move occurs on the left side of the tree. The time complexity for the best-case order is $O(b^{m/2})$ (since we search only the left sub-tree, not the right subtree).

2.9 SOLUTIONS/ANSWERS

Check your progress 1:

Answer 1: A number of factors need to be taken into consideration when developing a state-space representation. Factors that must be addressed are:

- What is the goal to be achieved?
- What are the legal moves or actions?
- What knowledge needs to be represented in the state description?
- Type of problem - There are basically three types of problems. Some problems only need a representation, e.g., crossword puzzles. Other problems require a yes or no response, indicating whether a solution can be found or not. Finally, the last type of problems are those that require a solution path as an output, e.g., mathematical theorems, Towers of Hanoi. In these cases, we know the goal state, and we need to know how to attain this state

- Best solution vs. Good enough solution - For some problems, a good enough solution is sufficient. For example: theorem proving eight squares.
However, some problems require a best or optimal solution, e.g., the traveling salesman problem.

Answer 2

(a) Formulation of the Missionaries and Cannibals problem:

State: (#M, #C, 0/1)

Where #M represents the number of missionaries in the left side bank (i.e., left side of the river)

#C: represents the number of cannibals in the left side bank (i.e., left side of the river)

0/1: indicate the boat position of the boat. 0 indicates the boat is on the left side of the river, and 1 indicates the boat is on the right side.

Start state:(3,3,0)

Goal State: (0,0,1)

Operator: State will be changed by moving missionaries and/or cannibals from one side to another using a boat. So, it can be represented as the number of persons on either side of the river. Note that the boat can carry a maximum of 2 persons.

Boat carries: (1,0) or (0,1) or (1,1) or (2,0) or (0,2). Here in (i, j), i represents the number of missionaries and j means number of cannibals.

(b) Solution of the Missionaries and Cannibals problem:

Start state: (3,3,0)

Goal State: (0,0,1)

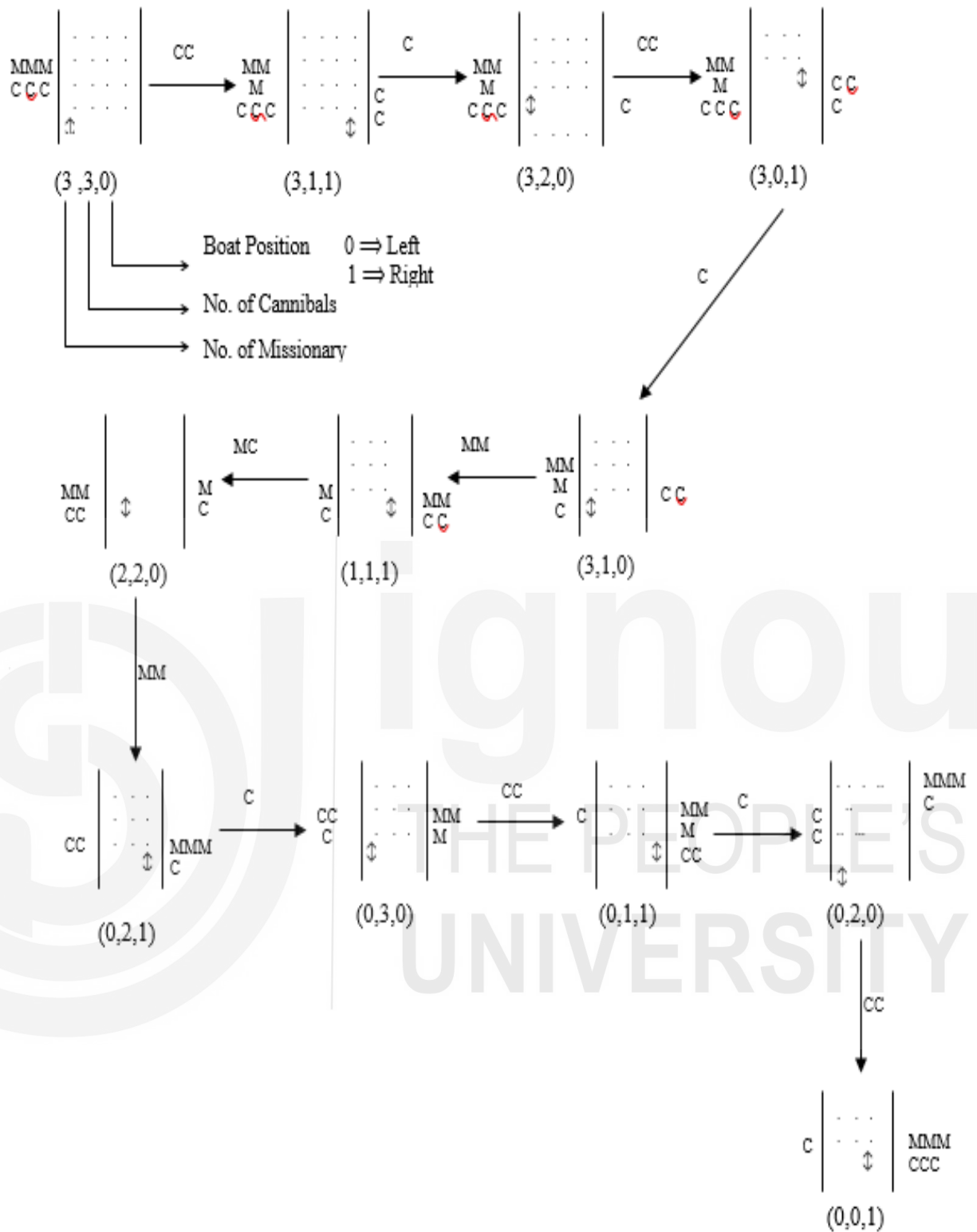


Figure: Solution of the Missionaries and Cannibals problem

A state space tree for this problem is shown below:

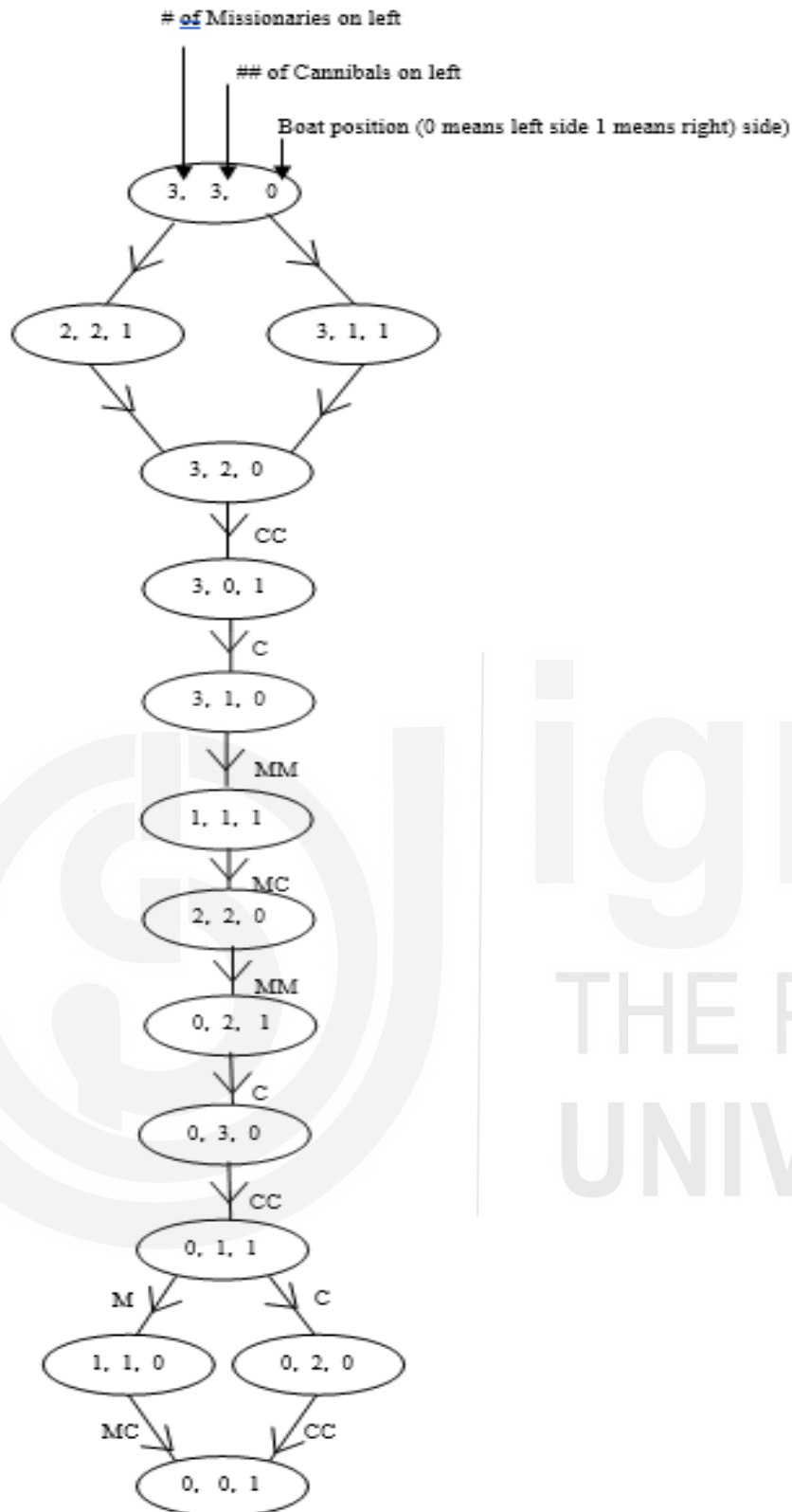


Figure: A state space tree showing all possible solutions of the missionaries and cannibal problem.

Answer 3: Towers Hanoi A possible state space representation of the Towers Hanoi problem using a graph is indicated in **Figure 1**.

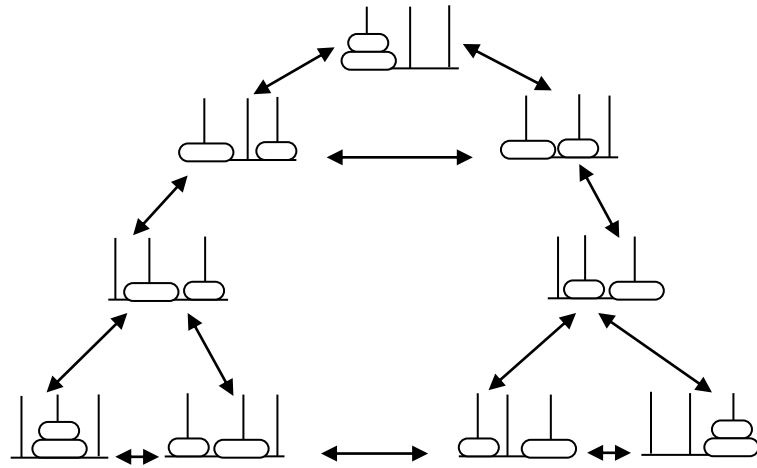
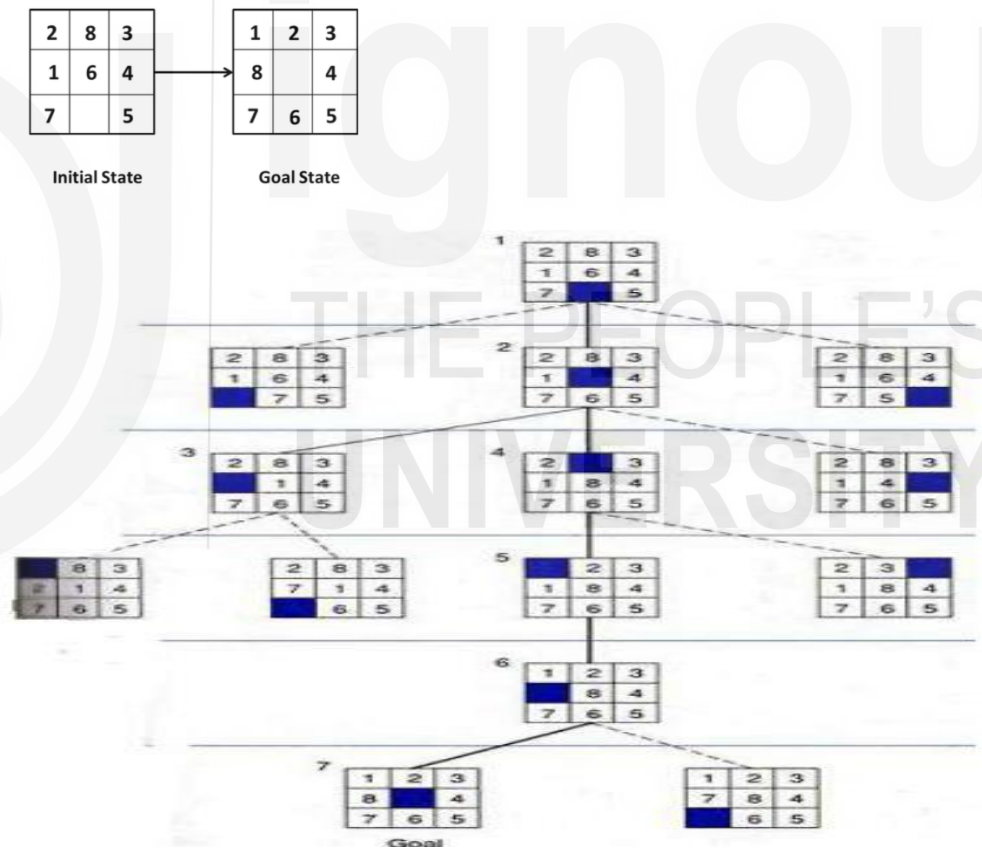


Figure 1: Towers of Hanoi state space representation for n=2

The legal moves in this state space involve moving one ring from one pole to another, moving one ring at a time, and ensuring that a larger ring is not placed on a smaller ring.

Answer 4:



Minimum Cost path for solution= 6

Answer 5: Backtracking Algorithm: The idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for clashes with already placed queens. In the current column, if we find a row for which there is no clash, we mark this row and column as part of the solution. If we do not find such a row due to clashes, then we backtrack and return false.

- 1) Start in the leftmost column
- 2) If all queens are placed
return true
- 3) Try all rows in the current column.
Do following for every tried row.

a) If the queen can be placed safely in this row then mark this [row, column] as part of the solution and recursively check if placing queen here leads to a solution.

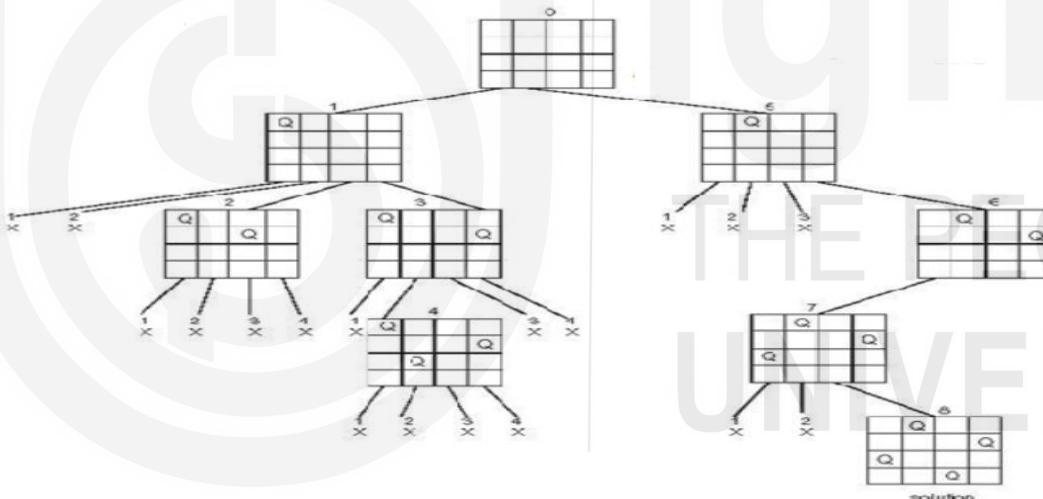
b) If placing the queen in [row, column] leads to a solution then return true.

c) If placing queen doesn't lead to a solution then unmark this [row, column] (Backtrack) and go to step (a) to try other rows.

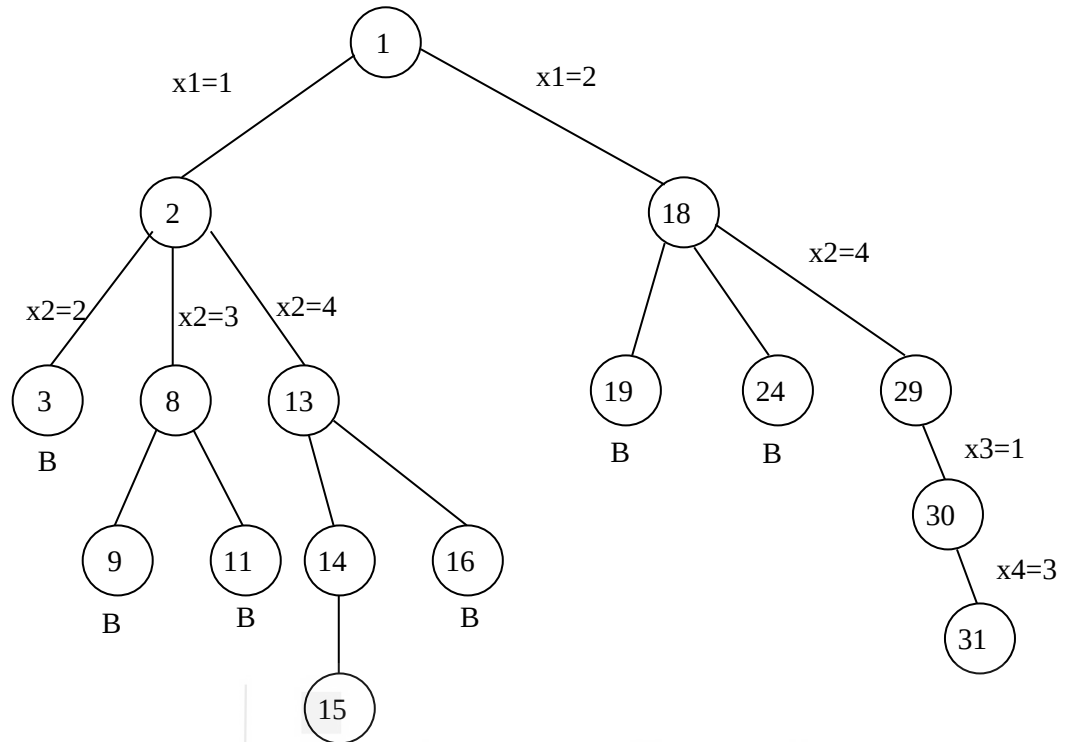
4) If all rows have been tried and nothing worked, return false to trigger backtracking.

State space tree for 4 Queen's problem

A state-space tree (SST) can be constructed to show the solution to this problem. The following SST (figure 1) shows one possible solution $\{x_1, x_2, x_3, x_4\} = \{2, 4, 3, 1\}$ for the 4 Queen's Problem.



Or we can also denote the State space tree as follows:



B denotes the Dead Node (nonpromising node). The figure 1 shows the Implicit tree for 4 queen problem for solution $\langle 2, 4, 1, 3 \rangle$. The Root represents an initial state. The Nodes reflect the specific choices made for the components of a solution. Explore The state space tree using **depth-first search**. "Prune" non-promising nodesdfs stops exploring subtree rooted at nodes leading to no solutions and then backtracks to its parent node

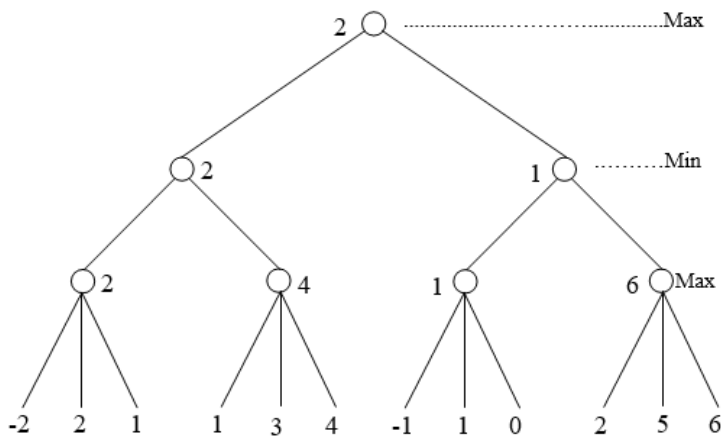
Check your progress 2

Answer1:

Solution: Alpha-beta pruning is an advance version of MINIMAX algorithm. The drawback of minimax strategy is that it explores each node in the tree deeply to provide the best path among all the paths. This increases its time complexity. If b is the branching factor and d is the depth of the tree, then time complexity of MINIMAX algorithm is $O(b^d)$ that is exponential.

Alpha-Beta pruning is a way of finding the optimal Minimax solution while avoiding searching subtrees of moves which won't be selected. The effectiveness of Alpha-beta pruning is highly dependent on the order in which each node is examined. **The** ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best move occur at the left side of the tree. We apply DFS hence it first search left of the tree and go deep twice as minimax algorithm in the same amount of time. The time complexity for best case order is $O(b^{m/2})$ (since we search only left sub tree, not a right subtree).

Answer 2: The final game tree with max and min value at each node is shown in the following figure.



Answer3:

Solution: Solve the question as shown in Example1

The initial call starts from A. We initially start the search by setting the initial value of $\alpha = -\infty$ and $\beta = +\infty$ for root node A. These values are passed down to subsequent nodes in the tree. At A the maximizer must choose max of B and C, so A calls B first. At B it the minimizer must choose min of D and E and hence calls D first. At D, it looks at its left child which is a leaf node. This node returns a value of 3. Now the value of alpha at D is $\max(-\infty, 3)$ which is 3. To decide whether it's worth looking at its right node or not, it checks the condition $\alpha \geq \beta$. This is false since $\beta = +\infty$ and $\alpha = 3$. So, it continues the search.

D now looks at its right child which returns a value of 5. At D, $\alpha = \max(3, 5)$ which is 5. Now the value of node D is 5. Value at node D=5, move up to node B(=5). Now at node B, β value will be modified as

$$\beta = \min(+\infty, 5) = 5.$$

B now calls E, we pass the α and β value from top node B to bottom node E as $[\alpha = -\infty, \beta = 5]$. Since, node E is at MAX level, so only α value will be change. Now, at E, it first checks the left child (which is a terminal node) with value 6. This node returns a value of 6.

Now, the value of α at node E is calculated as $\alpha = \max(-\infty, 6) = 6$, so value of Node(E)=6 and modified value of α and β at node E is $[\alpha = 6, \beta = 5]$. To decide whether it's worth looking at its right node or not, we check $\alpha \geq \beta$. The answer is **YES**, since $6 \geq 5$. So, we prune (cut) the right branch of E, as shown in figure (a).

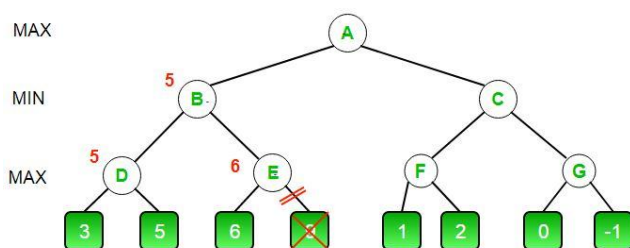


Figure (a) game tree after applying alpha-beta pruning on left side of node A

Similarly, we solve for right sub tree for Node A [refer the example 1 and solve for right sub tree part]. The final tree with node value at every node is shown in the figure(b).

Thus finally, α and β value at node A is $[\alpha = 5, \beta = +\infty]$ and best value at Node(A) = $\max(5, 2) = 5$. So, optimal value for the maximizer is 5 and there are 3 terminal nodes are pruned (9, 0 and -1). The optimal search path is $A \rightarrow B \rightarrow D \rightarrow 5$, as shown in figure (b).

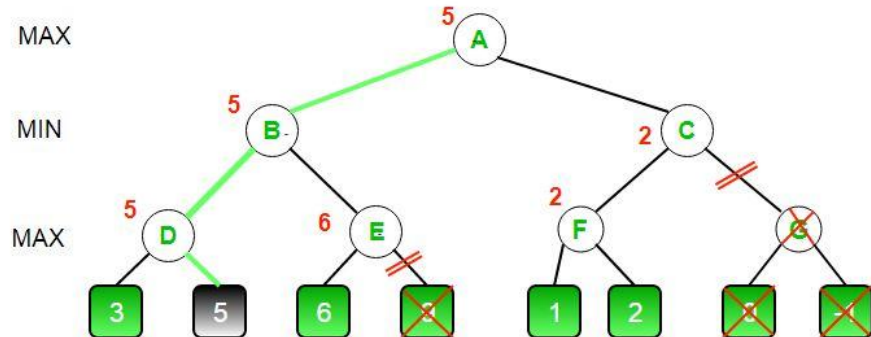
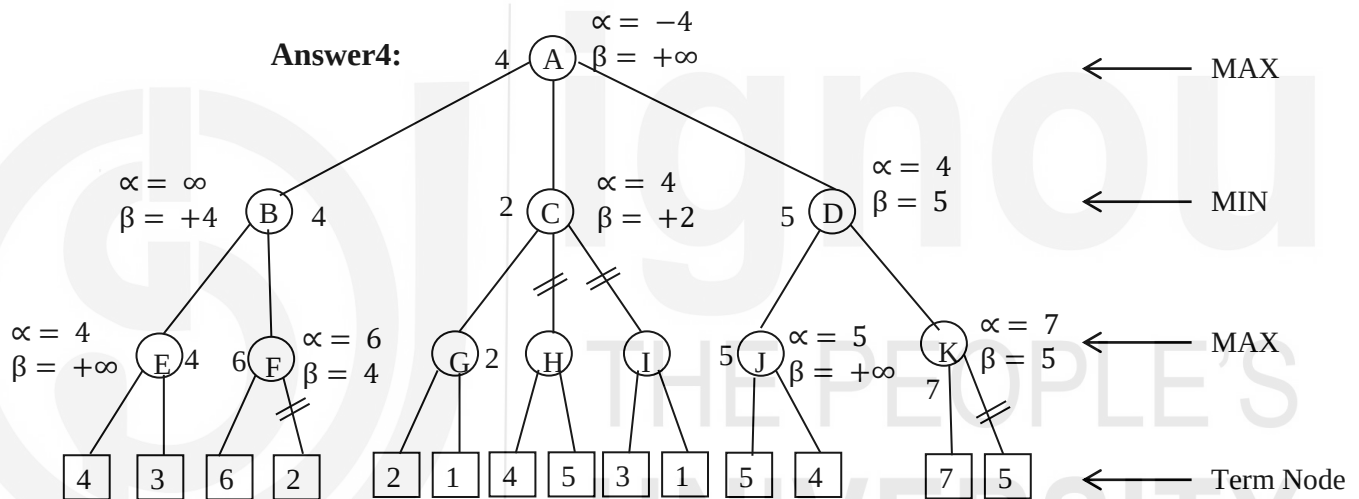


Figure (b) Final tree with node value on every node with prune branches



Answer5: 5

Answer6: 7

Answer 7: Option (B) 17

2.11 FURTHER READINGS

1. Ela Kumar, "Artificial Intelligence", IK International Publications
2. E. Rich and K. Knight, "Artificial intelligence", Tata Mc Graw Hill Publications
3. N.J. Nilsson, "Principles of AI", Narosa Publ. House Publications
4. John J. Craig, "Introduction to Robotics", Addison Wesley publication
5. D.W. Patterson, "Introduction to AI and Expert Systems" Pearson publication